Université de Gafsa

Institut Supérieur des Sciences Appliquées et de Technologie de Gafsa

Département Informatique

RAPPORT DE STAGE

Conception et Développement d'un Système Automatisé d'Auto-Défense Réseau (ANSDS)

Entreprise d'accueil

Agence Nationale de la Cybersécurité (ANCS)

Auteur : Yahya Imen

Encadrant professionnel : M. Bengayed Wajdi

Année universitaire 2025–2026

Remerciements

Je tiens à exprimer ma profonde gratitude à l'Agence Nationale de la Cybersécurité (ANCS) pour m'avoir accueilli durant ce stage et m'avoir offert un environnement propice à l'apprentissage et à la réalisation de ce projet.

Mes sincères remerciements s'adressent particulièrement à mon encadrant professionnel, M. Bengayed Wajdi, pour son accompagnement, ses conseils, sa disponibilité et la confiance qu'il m'a accordée tout au long de ce travail.

Je remercie également l'Institut Supérieur des Sciences Appliquées et de Technologie de Gafsa, ainsi que l'ensemble des enseignants du Département Informatique, pour la qualité de la formation reçue.

Enfin, j'adresse mes remerciements à toutes les personnes ayant contribué, de près ou de loin, à la réussite de ce projet de stage.

Résumé

Ce rapport présente la conception et le développement d'un système automatisé d'auto-défense réseau, intitulé *Automated Network Self-Defense System* (ANSDS). Ce projet a été réalisé dans le cadre d'un stage d'été au sein de l'Agence Nationale de la Cybersécurité (ANCS).

L'objectif principal est de concevoir un système capable d'analyser le trafic réseau, de détecter automatiquement des comportements suspects, de transmettre des alertes de sécurité en temps réel et d'appliquer une réponse automatique sur l'infrastructure réseau.

Une plateforme expérimentale a été développée sous GNS3 en intégrant un routeur Cisco, un commutateur, un serveur Ubuntu, une machine Kali Linux ainsi qu'un accès Internet. Cet environnement permet de reproduire différents scénarios d'activité réseau et de valider progressivement les fonctionnalités du système.

Le moteur de détection a été développé en Python avec la bibliothèque Scapy. Il permet notamment d'analyser les tentatives de connexion SSH et de détecter des comportements associés aux scans réseau Nmap. La détection prend en compte plusieurs indicateurs tels que les signatures TCP, les ports ciblés, la répétition des paquets et la fenêtre temporelle.

Le système intègre également un mécanisme de corrélation avec l'activité SSH afin de renforcer la décision de sécurité. Lorsqu'un événement suspect est confirmé, une notification est automatiquement transmise via Telegram. Le module Netmiko permet ensuite d'interagir avec le routeur Cisco afin d'appliquer dynamiquement une ACL de blocage.

Le blocage est temporaire et sa durée est configurée par le système. À son expiration, l'ACL est automatiquement supprimée afin de rétablir l'état initial du réseau. Un mécanisme de journalisation permet également de conserver une trace des principales étapes du traitement.

Les expérimentations réalisées ont permis de valider la détection des scans Nmap SYN, FIN, NULL et Xmas, la corrélation avec l'activité SSH, l'envoi des alertes Telegram, l'application du blocage Cisco ainsi que le déblocage automatique.

Mots-clés : Cybersécurité, ANSDS, SSH, Nmap, Scapy, GNS3, Cisco IOS, Netmiko, Telegram, détection réseau, auto-défense réseau.

Table des matières

Résumé **ii**

1 Introduction Générale **1**
1.1 Contexte du projet 1
1.2 Problématique 1
1.3 Objectifs du projet 2
1.4 Méthodologie adoptée 2
1.5 Technologies et outils utilisés 3
1.6 Organisation du rapport 4

2 Conception du Laboratoire Réseau **5**
2.1 Introduction 5
2.2 Objectifs du laboratoire 5
2.3 Architecture générale du laboratoire 6
2.4 Équipements du laboratoire 6
2.4.1 Routeur Cisco 7
2.4.2 Commutateur Cisco 8
2.4.3 Serveur Ubuntu 8
2.4.4 Machine Kali Linux 8
2.5 Plan d'adressage IP 8
2.6 Configuration NAT 9
2.7 Validation de l'environnement réseau 10
2.7.1 Version Cisco IOS 10
2.7.2 Validation de la connectivité Internet 10
2.8 Validation des communications réseau 10
2.8.1 Communication entre le serveur Ubuntu et le routeur 10
2.8.2 Communication entre Kali Linux et le routeur 12
2.8.3 Communication entre le serveur Ubuntu et Kali Linux 12
2.8.4 Communication entre Kali Linux et le serveur Ubuntu 12
2.9 Validation du service SSH 13
2.9.1 Installation du serveur OpenSSH 13
2.9.2 Vérification de l'état du service 14
2.9.3 Vérification du port d'écoute 14
2.9.4 Validation d'une connexion SSH 14
2.10 Conclusion 14

3 Développement du moteur de détection SSH avec Scapy 16
3.1 Introduction . . . 16
3.2 Objectifs de la phase . . . 16
3.3 Étude du protocole SSH . . . 17
3.3.1 Établissement d'une connexion SSH . . . 17
3.3.2 Analyse du Three-Way Handshake TCP . . . 17
3.3.3 Informations observables malgré le chiffrement . . . 19
3.4 Développement progressif du moteur de détection . . . 20
3.4.1 Première capture réseau avec Scapy . . . 20
3.4.2 Filtrage du trafic SSH . . . 21
3.4.3 Détection des paquets TCP SYN . . . 22
3.4.4 Comptage des tentatives de connexion . . . 23
3.4.5 Détection par seuil . . . 23
3.4.6 Fenêtre temporelle glissante . . . 23
3.5 Validation expérimentale . . . 24
3.6 Analyse des résultats . . . 25
3.7 Conclusion . . . 25

4 Développement de la notification et de la réponse automatisée 26
4.1 Introduction . . . 26
4.2 Objectifs du chapitre . . . 26
4.3 Développement du système de notification Telegram . . . 27
4.3.1 Présentation de Telegram Bot API . . . 27
4.3.2 Création du Bot Telegram . . . 27
4.3.3 Développement du module de notification . . . 30
4.3.4 Intégration avec le moteur de détection SSH . . . 31
4.3.5 Construction d'une alerte professionnelle . . . 32
4.4 Validation du système de notification . . . 35
4.4.1 Validation expérimentale . . . 35
4.4.2 Analyse des résultats . . . 35
4.5 Développement du mécanisme de réponse automatique . . . 36
4.5.1 Préparation du routeur Cisco . . . 36
4.5.2 Installation et configuration de Netmiko . . . 36
4.5.3 Développement du module Cisco . . . 37
4.5.4 Gestion des exceptions . . . 38
4.5.5 Intégration avec le moteur de détection . . . 38
4.5.6 Validation expérimentale . . . 39
4.6 Analyse des résultats . . . 39
4.7 Conclusion . . . 40

5 Intégration et automatisation de la réponse sur le routeur Cisco 41
5.1 Introduction . . . 41
5.2 Objectifs de la phase . . . 42
5.3 Préparation du routeur Cisco . . . 42
5.3.1 Configuration SSH du routeur . . . 42
5.3.2 Validation de la connexion SSH . . . 43
5.4 Installation et configuration de Netmiko . . . 43
5.4.1 Installation de la bibliothèque . . . 44
5.4.2 Premier test de connexion . . . 44

5.4.3 Récupération automatique des informations du routeur 45
5.5 Développement du module Cisco 45
5.5.1 Architecture modulaire 45
5.5.2 Création du module `cisco_acl_manager.py` 46
5.5.3 Création dynamique d'une ACL 46
5.5.4 Suppression automatique d'une ACL 47
5.6 Gestion des exceptions 47
5.6.1 Gestion des erreurs de communication 47
5.6.2 Utilisation des blocs `try`, `except` et `finally` 48
5.6.3 Validation du mécanisme de gestion des exceptions 48
5.7 Intégration avec le moteur de détection 48
5.7.1 Transmission de l'adresse IP détectée 48
5.7.2 Enchaînement automatique des modules 49
5.7.3 Architecture finale du système 49
5.8 Validation expérimentale 50
5.8.1 Validation de la communication avec le routeur Cisco 50
5.8.2 Validation de la création dynamique d'une ACL 50
5.8.3 Validation de la suppression automatique d'une ACL 50
5.8.4 Validation de la robustesse du module Cisco 51
5.8.5 Validation de la chaîne complète de détection et de réponse 51
5.9 Analyse des résultats 52
5.10 Conclusion 52

6 Détection des scans réseau et corrélation des événements 53
6.1 Introduction 53
6.2 Objectifs de la phase 53
6.3 Évolution de la stratégie de détection 54
6.3.1 Limites de la détection SSH initiale 54
6.3.2 Introduction de la détection Nmap 54
6.4 Signatures TCP des scans Nmap 55
6.4.1 TCP SYN Scan 55
6.4.2 TCP FIN Scan 56
6.4.3 TCP NULL Scan 56
6.4.4 TCP Xmas Scan 57
6.5 Détection comportementale des scans 58
6.6 Réduction des faux positifs 59
6.6.1 Validation du trafic légitime 59
6.7 Corrélation avec l'activité SSH 59
6.7.1 Scan sans corrélation SSH 60
6.8 Notification de sécurité 61
6.9 Réponse automatique Cisco 61
6.9.1 Activation du blocage temporaire 62
6.9.2 Application de l'ACL Cisco 63
6.10 Journalisation des événements 64
6.11 Expiration automatique du blocage 65
6.11.1 Suppression de l'ACL 65
6.12 Architecture finale de la Phase 5 66
6.13 Validation expérimentale 66

6.14 Analyse des résultats 67
6.15 Journalisation et traçabilité 67
6.16 Conclusion 68

7 Conclusion générale et perspectives **69**
7.1 Bilan général du projet 69
7.2 Principales contributions réalisées 69
7.3 Bilan de la chaîne de défense développée 70
7.4 Limites du système 70
7.5 Perspectives d'évolution 71
7.6 Bilan du stage 71

Table des figures

2.1 Architecture générale du laboratoire réseau développé sous GNS3 6
2.2 Configuration matérielle du routeur Cisco 7
2.3 Configuration IP des interfaces du routeur Cisco 7
2.4 Configuration réseau du serveur Ubuntu 8
2.5 Configuration réseau de la machine Kali Linux 9
2.6 Configuration NAT du routeur Cisco 9
2.7 Vérification de la version Cisco IOS 11
2.8 Validation de la connectivité Internet 11
2.9 Test de connectivité entre le serveur Ubuntu et le routeur Cisco 12
2.10 Test de connectivité entre Kali Linux et le routeur Cisco 12
2.11 Communication entre le serveur Ubuntu et Kali Linux 13
2.12 Communication entre Kali Linux et le serveur Ubuntu 13
2.13 Installation du serveur OpenSSH 14
2.14 Vérification de l'état du service SSH 14
2.15 Vérification de l'écoute du port TCP 22 14
2.16 Connexion SSH réussie entre Kali Linux et le serveur Ubuntu 15

3.1 Vue générale d'une connexion SSH 18
3.2 Paquet TCP SYN 18
3.3 Paquet TCP SYN/ACK 19
3.4 Paquet TCP ACK 19
3.5 Première version du détecteur développé avec Scapy 21
3.6 Capture des paquets réseau avec Scapy 21
3.7 Filtrage des paquets SSH 22
3.8 Détection des paquets TCP SYN 22
3.9 Comptage des tentatives de connexion 23
3.10 Déclenchement de l'alerte après dépassement du seuil 23
3.11 Détection d'une attaque après application de la fenêtre temporelle 24

4.1 Création du bot Telegram avec BotFather 28
4.2 Récupération du Chat ID 29
4.3 Validation du premier message envoyé via Telegram Bot API 31
4.4 Validation du module Python de notification 31
4.5 Intégration du module Telegram au moteur de détection SSH 32
4.6 Notification de sécurité générée automatiquement par l'ANSDS 34

5.1 Connexion SSH manuelle entre le serveur Ubuntu et le routeur Cisco . . . 43
5.2 Installation de la bibliothèque Netmiko 44
5.3 Première connexion établie avec Netmiko 44
5.4 Récupération automatique des informations du routeur Cisco 45

5.5 Création dynamique d'une ACL avec Netmiko 46
5.6 Suppression automatique d'une ACL 47
5.7 Validation de la gestion des exceptions Netmiko 48
5.8 Validation complète de la chaîne de détection et de réponse automatique . 51

6.1 Détection d'un scan Nmap SYN 55
6.2 Détection d'un scan Nmap FIN 56
6.3 Détection d'un scan Nmap NULL 57
6.4 Détection d'un scan Nmap Xmas 58
6.5 Trafic réseau légitime observé durant la validation 59
6.6 Corrélation entre un scan Nmap et une activité SSH suspecte 60
6.7 Scan Nmap sans corrélation avec une activité SSH 61
6.8 Activation du blocage temporaire 63
6.9 Application de l'ACL de blocage sur le routeur Cisco 64
6.10 Suppression automatique de l'ACL après expiration du blocage 65

Liste des tableaux

2.1 Plan d'adressage IP du laboratoire 9

Chapitre 1

Introduction Générale

1.1 Contexte du projet

L'évolution constante des infrastructures informatiques et leur interconnexion croissante exposent les réseaux à différentes formes de menaces. Les attaques peuvent notamment viser les services accessibles sur le réseau afin d'identifier les équipements, d'explorer les services disponibles ou de tenter d'obtenir un accès non autorisé.

Dans ce contexte, la surveillance du trafic réseau constitue une composante importante de la sécurité des infrastructures. Cependant, la simple observation des événements réseau ne suffit pas toujours à assurer une protection efficace. Un système de défense doit également être capable d'identifier les comportements suspects, d'alerter rapidement l'administrateur et, lorsque cela est nécessaire, d'appliquer automatiquement une mesure de protection.

C'est dans cette perspective que s'inscrit le projet *Automated Network Self-Defense System* (ANSDS), réalisé dans le cadre d'un stage d'été au sein de l'Agence Nationale de la Cybersécurité (ANCS). Le projet vise à concevoir progressivement un système capable d'analyser le trafic réseau, de détecter différents comportements malveillants et d'automatiser une partie de la réponse de sécurité.

L'environnement expérimental a été développé sous GNS3 [3] et repose notamment sur un routeur Cisco [2], un commutateur, un serveur Ubuntu et une machine Kali Linux. Le serveur Ubuntu représente la machine cible tandis que Kali Linux est utilisée pour reproduire différents scénarios d'activité offensive. Cette infrastructure fournit un environnement contrôlé permettant de développer et de valider les différents composants du système.

1.2 Problématique

La détection d'une activité malveillante sur un réseau présente plusieurs difficultés. Un événement réseau isolé ne constitue pas nécessairement une preuve suffisante d'une attaque. Par exemple, une tentative de connexion TCP vers un service SSH peut correspondre à une activité légitime. Une stratégie de détection efficace doit donc prendre en compte le comportement global d'une source ainsi que plusieurs indicateurs observables.

Dans les premières phases du projet, la surveillance des paquets TCP destinés au port SSH a permis de détecter des comportements caractérisés par une répétition importante de tentatives de connexion dans une fenêtre temporelle donnée. Cette approche a ensuite été

étendue afin de prendre en compte les comportements de reconnaissance réseau associés aux scans Nmap [5].

Le système analyse notamment les signatures TCP, les ports ciblés, la répétition des paquets et la fenêtre temporelle. La corrélation avec l'activité SSH permet également de renforcer la décision de sécurité avant de déclencher une réponse automatique.

La problématique principale du projet peut ainsi être formulée comme suit :

> *Comment concevoir un système automatisé capable de détecter des comportements réseau suspects, de réduire les décisions basées sur des événements isolés, de corréler plusieurs indicateurs de sécurité et de déclencher automatiquement une réponse adaptée sur l'infrastructure réseau ?*

Cette problématique implique plusieurs aspects complémentaires : l'analyse du trafic réseau, la détection comportementale, la corrélation des événements, la notification de sécurité et l'automatisation de la réponse réseau.

1.3 Objectifs du projet

L'objectif général du projet est de concevoir et de développer un système automatisé d'auto-défense réseau capable d'assurer une chaîne de traitement allant de l'observation du trafic jusqu'à l'application d'une mesure de protection.

Les objectifs spécifiques sont les suivants :

— mettre en place un laboratoire réseau contrôlé et reproductible sous GNS3 [3] ;
— configurer et valider les communications entre les différents équipements ;
— déployer et valider le service SSH sur le serveur Ubuntu ;
— analyser les caractéristiques observables du trafic TCP associé au protocole SSH [4, 10] ;
— développer un moteur de capture et d'analyse réseau avec Scapy [7] ;
— détecter les comportements suspects liés aux tentatives de connexion SSH ;
— utiliser une fenêtre temporelle et une logique comportementale afin de limiter les détections basées sur des paquets isolés ;
— détecter plusieurs types de scans réseau Nmap, notamment SYN, FIN, NULL et Xmas [5] ;
— corréler les événements de reconnaissance réseau avec une activité SSH suspecte ;
— transmettre automatiquement une notification de sécurité via Telegram [8] ;
— automatiser la communication avec le routeur Cisco à l'aide de Netmiko [1] ;
— appliquer dynamiquement une ACL permettant de bloquer temporairement une adresse IP identifiée comme malveillante [2] ;
— supprimer automatiquement la règle de blocage après expiration de la durée configurée ;
— journaliser les événements importants afin d'assurer la traçabilité du traitement.

1.4 Méthodologie adoptée

Le développement du système a été réalisé selon une approche progressive. Chaque fonctionnalité a d'abord été étudiée et développée séparément, puis validée expérimentalement avant son intégration avec les composants précédents.

La première étape a consisté à construire le laboratoire réseau sous GNS3 [3] et à vérifier la connectivité entre les différents équipements. Le routeur Cisco assure le routage

et l'accès au réseau externe, tandis que le serveur Ubuntu et la machine Kali Linux constituent les principales machines utilisées pour les expérimentations.

La deuxième étape a porté sur l'étude du trafic SSH et sur le développement du moteur de détection avec Scapy [7]. Le système a progressivement évolué de la simple capture de paquets vers le filtrage du trafic SSH, la détection des paquets TCP SYN, le comptage des tentatives et l'utilisation d'une fenêtre temporelle glissante.

La troisième étape a permis d'ajouter un mécanisme de notification avec Telegram [8]. Le module de notification a été conçu séparément du moteur de détection afin de conserver une architecture modulaire et de faciliter son intégration avec les futurs mécanismes de réponse.

La quatrième étape a porté sur l'automatisation du routeur Cisco avec Netmiko [1]. Cette intégration permet au système d'appliquer dynamiquement une ACL correspondant à l'adresse IP identifiée comme source d'une activité suspecte.

Enfin, la cinquième étape a étendu le mécanisme de détection aux scans réseau Nmap [5]. Quatre signatures TCP ont été étudiées : SYN, FIN, NULL et Xmas. La décision de sécurité ne repose pas uniquement sur un paquet isolé, mais sur l'analyse du comportement d'une même source, du nombre de ports ciblés et de la temporalité des événements.

Une corrélation avec l'activité SSH permet ensuite de renforcer la décision avant le déclenchement de la réponse automatisée.

La validation a été effectuée progressivement au moyen de scénarios contrôlés permettant de vérifier individuellement les différentes fonctionnalités puis leur enchaînement dans la chaîne complète de défense.

1.5 Technologies et outils utilisés

La réalisation du projet s'appuie sur plusieurs technologies complémentaires.

- **GNS3** est utilisé pour construire et reproduire l'environnement réseau expérimental [3].
- **Cisco IOS** est utilisé sur le routeur chargé du routage, du NAT et de l'application des ACL destinées au blocage des adresses IP [2].
- **Ubuntu Server** constitue la machine cible et héberge notamment le service OpenSSH [10].
- **Kali Linux** est utilisée comme machine d'expérimentation permettant de reproduire les différents comportements réseau étudiés.
- **Wireshark** est utilisé pour l'observation et l'analyse du trafic réseau [9].
- **Python** constitue le langage principal de développement du système [6].
- **Scapy** permet de capturer et d'analyser les paquets réseau afin d'identifier les signatures et comportements recherchés [7].
- **Telegram Bot API** est utilisée pour transmettre les alertes de sécurité [8].
- **Netmiko** permet d'automatiser les interactions avec le routeur Cisco et d'appliquer les mesures de protection réseau [1].

L'utilisation combinée de ces technologies permet de construire une architecture dans laquelle chaque composant possède une responsabilité clairement définie.

1.6 Organisation du rapport

Le présent rapport est organisé en sept chapitres correspondant aux principales étapes de conception, de développement, d'intégration et d'évolution du système Automated Network Self-Defense System (ANSDS).

Le chapitre 1, intitulé *Introduction Générale*, présente le contexte du projet, la problématique étudiée, les objectifs fixés, la méthodologie adoptée ainsi que les principales technologies utilisées.

Le chapitre 2, intitulé *Conception du Laboratoire Réseau*, présente la mise en place de l'environnement expérimental sous GNS3. Il décrit l'architecture du laboratoire, les équipements utilisés, le plan d'adressage IP, la configuration du NAT ainsi que les différentes validations de connectivité et du service SSH.

Le chapitre 3, intitulé *Développement du moteur de détection SSH avec Scapy*, présente l'étude du trafic SSH et la construction progressive du moteur de détection. Il décrit notamment la capture des paquets, le filtrage du trafic SSH, la détection des paquets TCP SYN, le comptage des tentatives et l'utilisation d'une fenêtre temporelle glissante.

Le chapitre 4, intitulé *Développement de la notification et de la réponse automatisée*, présente la mise en place du système de notification Telegram ainsi que les premiers mécanismes de réponse automatisée. Il décrit notamment la création du bot Telegram, le développement du module de notification, l'intégration avec le moteur de détection, ainsi que la mise en place initiale de l'automatisation Cisco avec Netmiko.

Le chapitre 5, intitulé *Intégration et automatisation de la réponse sur le routeur Cisco*, approfondit l'intégration de Netmiko avec le routeur Cisco. Il présente la préparation du routeur, la récupération automatique de ses informations, le développement du module de gestion des ACL, la création et la suppression dynamiques des règles de blocage, la gestion des exceptions ainsi que la validation complète de la chaîne de détection et de réponse.

Le chapitre 6, intitulé *Détection des scans réseau et corrélation des événements*, présente l'évolution du système vers une stratégie de détection multi-indicateurs. Il traite des scans Nmap SYN, FIN, NULL et Xmas, de l'analyse comportementale, de la corrélation avec l'activité SSH, de la notification Telegram, du blocage temporaire sur Cisco, de l'expiration automatique du blocage et de la journalisation des événements.

Enfin, le chapitre 7, intitulé *Conclusion générale et perspectives*, présente le bilan global du projet, les principales contributions réalisées, les limites de la solution développée, les perspectives d'évolution ainsi que les principaux acquis du stage.

Chapitre 2

Conception du Laboratoire Réseau

2.1 Introduction

Avant de développer le système **Automated Network Self-Defense System (ANSDS)**, il a été indispensable de concevoir une plateforme de test permettant de reproduire un environnement réseau réaliste et entièrement maîtrisé. Ce laboratoire constitue la base expérimentale du projet et permet de développer, de tester et de valider chaque composant du système dans des conditions proches d'une infrastructure d'entreprise.

L'environnement a été réalisé à l'aide de GNS3 [3] en intégrant un routeur Cisco IOS [2], un commutateur Ethernet, un serveur Ubuntu, une machine Kali Linux ainsi qu'un accès Internet via un mécanisme de traduction d'adresses réseau (Network Address Translation – NAT). Cette architecture offre un environnement adapté à l'étude du protocole SSH [10], à la capture des paquets réseau ainsi qu'au développement des mécanismes de détection et de réponse automatique.

Avant de commencer les phases de détection d'attaques, l'ensemble des équipements a été configuré puis validé afin de garantir le bon fonctionnement des communications réseau.

2.2 Objectifs du laboratoire

La mise en place du laboratoire poursuit plusieurs objectifs techniques indispensables au développement du projet ANSDS :

- Concevoir une infrastructure réseau représentative d'un environnement réel.
- Déployer un serveur Ubuntu jouant le rôle de machine victime.
- Déployer une machine Kali Linux représentant l'attaquant.
- Configurer le routage et l'accès Internet à l'aide d'un routeur Cisco.
- Mettre en œuvre la traduction d'adresses (NAT).
- Valider l'ensemble des communications réseau avant le développement des mécanismes de sécurité.
- Préparer un environnement permettant l'analyse du trafic SSH avec Wireshark.
- Fournir une plateforme stable pour les futures phases de détection et de réponse automatique.

2.3 Architecture générale du laboratoire

L'architecture du laboratoire a été développée sous GNS3 [3] afin de reproduire une infrastructure réseau simple, stable et facilement reproductible.

Elle est constituée des éléments suivants :

— un routeur Cisco IOS assurant le routage des paquets et l'accès Internet ;
— un commutateur Cisco reliant les différentes machines du réseau local ;
— un serveur Ubuntu représentant la machine victime ;
— une machine Kali Linux représentant l'attaquant ;
— un nœud NAT permettant l'accès aux ressources Internet.

La Figure 2.1 présente l'architecture générale du laboratoire utilisée durant la totalité de la première phase du projet.

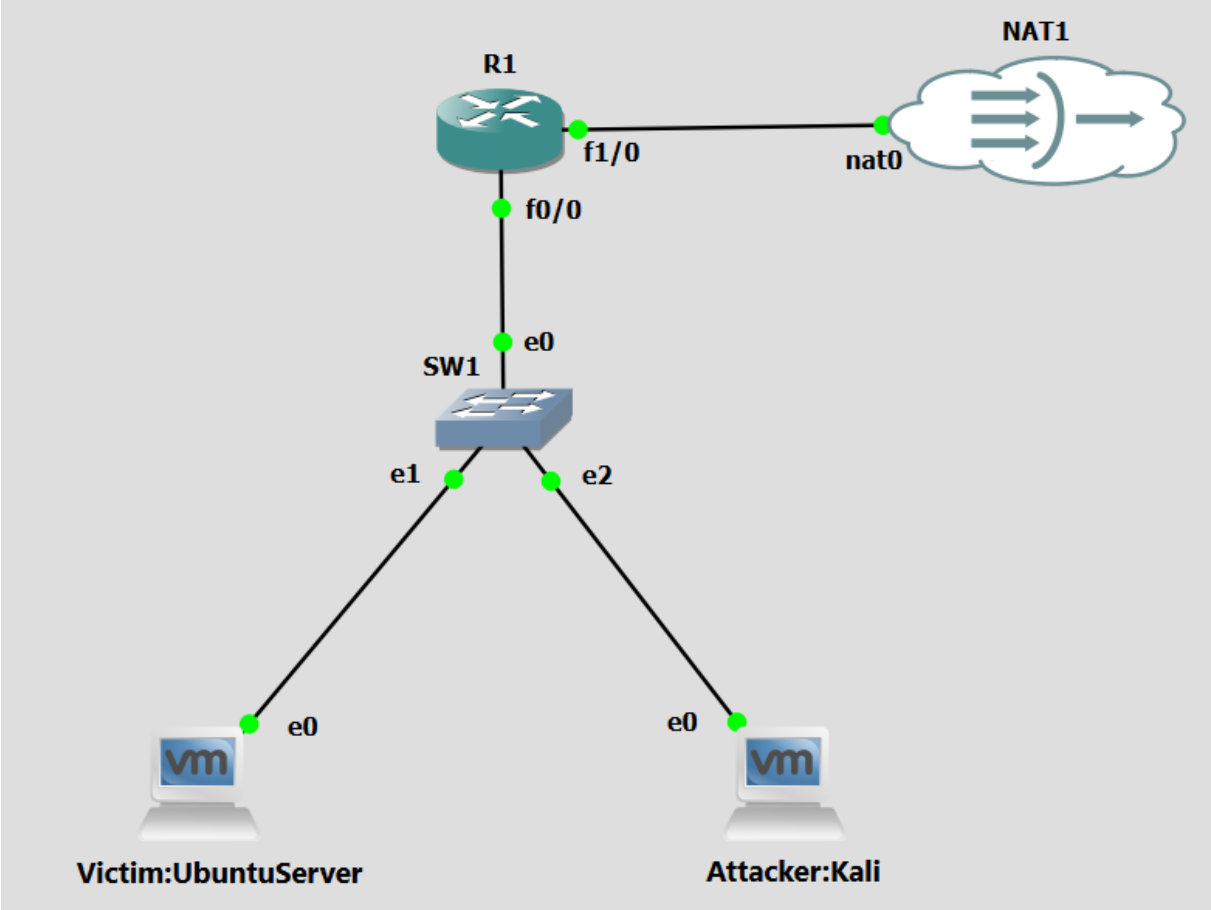


FIGURE 2.1 – Architecture générale du laboratoire réseau développé sous GNS3

Cette architecture permet d'isoler totalement l'environnement de test tout en offrant les fonctionnalités nécessaires au développement du système ANSDS. Elle constitue la base sur laquelle seront réalisées les captures réseau, les attaques SSH simulées ainsi que les mécanismes de détection et de défense présentés dans les chapitres suivants.

2.4 Équipements du laboratoire

L'infrastructure du laboratoire repose sur plusieurs équipements réseau ayant chacun une fonction bien définie. Ensemble, ils permettent de reproduire un environnement repré-

sentatif d'un réseau d'entreprise tout en offrant une plateforme adaptée au développement et à la validation du système ANSDS.

Les sous-sections suivantes présentent le rôle de chaque équipement ainsi que les principales configurations réalisées.

2.4.1 Routeur Cisco

Le routeur Cisco constitue l'élément central de l'architecture réseau. Il assure le routage des paquets entre le réseau local et Internet, ainsi que la mise en œuvre du mécanisme de traduction d'adresses (NAT). Il jouera également un rôle essentiel dans les phases suivantes du projet, puisqu'il sera chargé d'appliquer automatiquement des listes de contrôle d'accès (ACL) afin de bloquer les adresses IP identifiées comme malveillantes [2].

Le routeur utilisé est basé sur la plateforme Cisco 7200 et dispose de plusieurs interfaces FastEthernet permettant d'interconnecter le réseau local au nœud NAT de GNS3.

La Figure 2.2 présente la configuration matérielle retenue pour le routeur.

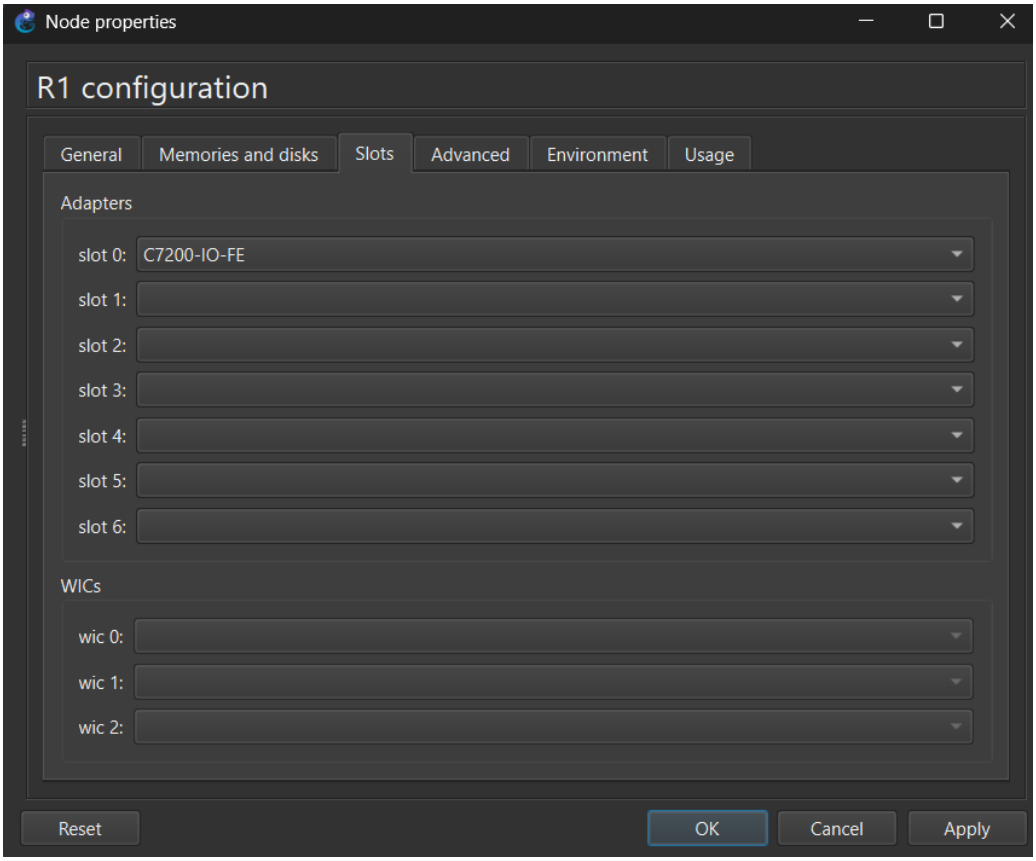


FIGURE 2.2 – Configuration matérielle du routeur Cisco

La configuration IP des interfaces du routeur est présentée à la Figure 2.3. L'interface FastEthernet0/0 est utilisée comme passerelle du réseau local tandis que l'interface FastEthernet1/0 assure la connexion vers Internet.

```
R1#show ip int br
Interface                  IP-Address      OK? Method Status                Protocol
FastEthernet0/0            192.168.10.1    YES manual up                    up
```

FIGURE 2.3 – Configuration IP des interfaces du routeur Cisco

2.4.2 Commutateur Cisco

Le commutateur Cisco assure l'interconnexion des équipements du réseau local. Il relie le serveur Ubuntu, la machine Kali Linux et le routeur Cisco au sein du même segment Ethernet.

Aucune configuration spécifique n'a été nécessaire sur ce commutateur. Son rôle est exclusivement de fournir la connectivité de niveau 2 entre les différents équipements du laboratoire. Comme présenté à la Figure 2.1, il constitue le point central des communications locales.

2.4.3 Serveur Ubuntu

Le serveur Ubuntu représente la machine victime du projet. Il héberge le service OpenSSH qui sera utilisé lors des différentes expérimentations de sécurité. Les futures attaques par force brute ainsi que les mécanismes de détection développés dans ANSDS cibleront cette machine.

Une configuration IP statique a été appliquée afin de garantir la stabilité de l'environnement de test. Le serveur utilise l'adresse IP 192.168.10.10/24 et la passerelle par défaut 192.168.10.1.

La configuration réseau du serveur est illustrée à la Figure 2.4.

```
imen@ubuntuserver:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:09:36:59 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.10.10/24 brd 192.168.10.255 scope global ens33
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe09:3659/64 scope link
       valid_lft forever preferred_lft forever
imen@ubuntuserver:~$ ip route
default via 192.168.10.1 dev ens33 proto static
192.168.10.0/24 dev ens33 proto kernel scope link src 192.168.10.10
```

FIGURE 2.4 – Configuration réseau du serveur Ubuntu

2.4.4 Machine Kali Linux

La machine Kali Linux représente l'attaquant dans le laboratoire. Elle est utilisée pour établir les connexions SSH, générer les attaques simulées et valider le fonctionnement des mécanismes de détection et de réponse automatique.

Comme le serveur Ubuntu, cette machine possède une configuration IP statique afin d'assurer la reproductibilité des expérimentations. Son adresse IP est 192.168.10.20/24 et sa passerelle par défaut est 192.168.10.1.

La Figure 2.5 présente la configuration réseau de la machine Kali Linux.

2.5 Plan d'adressage IP

Afin d'assurer une communication stable entre les différents équipements du laboratoire, un plan d'adressage IP statique a été défini. Chaque équipement possède une adresse IP fixe facilitant les phases de développement, de test et de validation.

Le Tableau 2.1 présente le plan d'adressage adopté.

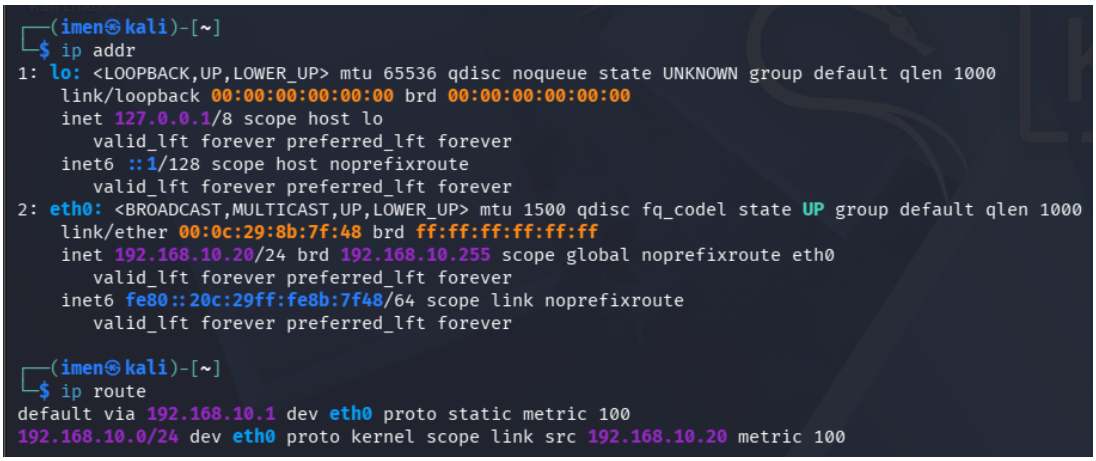


FIGURE 2.5 – Configuration réseau de la machine Kali Linux

TABLE 2.1 – Plan d'adressage IP du laboratoire

Équipement	Interface	Adresse IP	Passerelle
Routeur Cisco	FastEthernet0/0	192.168.10.1/24	—
Serveur Ubuntu	ens33	192.168.10.10/24	192.168.10.1
Kali Linux	eth0	192.168.10.20/24	192.168.10.1

L'ensemble des machines du réseau local appartient au sous-réseau `192.168.10.0/24`. Le routeur Cisco joue le rôle de passerelle par défaut et assure l'acheminement des paquets vers le réseau externe via le mécanisme de traduction d'adresses (NAT).

2.6 Configuration NAT

Afin de permettre aux machines virtuelles du laboratoire d'accéder au réseau Internet, une traduction d'adresses réseau (*Network Address Translation* – NAT) a été configurée sur le routeur Cisco [2].

Le mécanisme retenu est le *Port Address Translation* (PAT), également appelé surcharge NAT (*NAT Overload*). Cette technique permet à plusieurs machines du réseau privé de partager une seule adresse IP publique correspondant à l'interface connectée au réseau externe.

Dans cette configuration, l'interface FastEthernet0/0 est définie comme interface interne (*inside*), tandis que l'interface FastEthernet1/0 est configurée comme interface externe (*outside*).

La Figure 2.6 présente la configuration NAT appliquée sur le routeur Cisco.

```
R1#show run | section ip nat
 ip nat inside
 ip nat outside
ip nat inside source list 1 interface FastEthernet1/0 overload
R1#
```

FIGURE 2.6 – Configuration NAT du routeur Cisco

L'instruction `ip nat inside source list 1 interface FastEthernet1/0 overload` active le mécanisme PAT. Toutes les machines appartenant à la liste d'accès numéro 1 peuvent ainsi accéder au réseau externe en utilisant l'adresse IP de l'interface FastEthernet1/0.

Cette configuration constitue une étape essentielle du laboratoire, puisqu'elle garantit la connectivité Internet nécessaire à l'installation des paquets logiciels, aux mises à jour du système ainsi qu'aux différentes expérimentations réalisées durant le projet.

2.7 Validation de l'environnement réseau

Une fois la configuration des différents équipements terminée, plusieurs vérifications ont été réalisées afin de s'assurer du bon fonctionnement de l'environnement avant de débuter les expérimentations liées au système ANSDS.

Ces vérifications concernent notamment la version du système Cisco IOS ainsi que la connectivité Internet du laboratoire.

2.7.1 Version Cisco IOS

Le routeur utilisé dans le laboratoire exécute le système d'exploitation Cisco IOS. La vérification de la version installée permet de confirmer la compatibilité des fonctionnalités utilisées dans le cadre du projet, notamment le routage IP, la traduction d'adresses (NAT) et les listes de contrôle d'accès (ACL) [2].

La Figure 2.7 présente les informations relatives à la version du système Cisco IOS.

2.7.2 Validation de la connectivité Internet

Après la configuration du mécanisme NAT, un test de connectivité a été réalisé afin de vérifier que le routeur était capable d'accéder au réseau Internet.

La réussite de ce test confirme le bon fonctionnement du routage ainsi que de la traduction d'adresses configurée précédemment.

La Figure 2.8 présente le résultat de ce test.

La validation de ces deux éléments confirme que l'infrastructure réseau est correctement configurée et prête pour les phases de test et de développement du système ANSDS.

2.8 Validation des communications réseau

Une fois l'ensemble des équipements configurés, plusieurs tests de connectivité ont été réalisés afin de vérifier le bon fonctionnement de l'infrastructure réseau. Ces tests permettent de s'assurer que chaque machine peut communiquer correctement avec les autres équipements du laboratoire avant le déploiement des mécanismes de sécurité.

Les essais ont été réalisés à l'aide de la commande `ping`, qui permet de vérifier la disponibilité d'un équipement ainsi que la bonne configuration du routage.

2.8.1 Communication entre le serveur Ubuntu et le routeur

Le premier test consiste à vérifier que le serveur Ubuntu peut joindre la passerelle du réseau local.

```
R1#show version
Cisco IOS Software, 7200 Software (C7200-ADVIPSERVICESK9-M), Version 15.0(1)M, RELEASE SOFTWARE (fc2)
Technical Support: http://www.cisco.com/techsupport
Copyright (c) 1986-2009 by Cisco Systems, Inc.
Compiled Wed 30-Sep-09 07:48 by prod_rel_team

ROM: ROMMON Emulation Microcode
BOOTLDR: 7200 Software (C7200-ADVIPSERVICESK9-M), Version 15.0(1)M, RELEASE SOFTWARE (fc2)

R1 uptime is 2 minutes
System returned to ROM by unknown reload cause - suspect boot_data[BOOT_COUNT] 0x0, BOOT_COUNT 0, BOOTDATA 19
System image file is "tftp://255.255.255.255/unknown"


This product contains cryptographic features and is subject to United
States and local country laws governing import, export, transfer and
use. Delivery of Cisco cryptographic products does not imply
third-party authority to import, export, distribute or use encryption.
Importers, exporters, distributors and users are responsible for
compliance with U.S. and local country laws. By using this product you
agree to comply with applicable laws and regulations. If you are unable
to comply with U.S. and local laws, return this product immediately.

A summary of U.S. laws governing Cisco cryptographic products may be found at:
http://www.cisco.com/wwl/export/crypto/tool/stqrg.html

If you require further assistance please contact us by sending email to
export@cisco.com.

Cisco 7206VXR (NPE400) processor (revision A) with 491520K/32768K bytes of memory.
Processor board ID 4279256517
R7000 CPU at 150MHz, Implementation 39, Rev 2.1, 256KB L2 Cache
6 slot VXR midplane, Version 2.1

Last reset from power-on

PCI bus mb0_mb1 (Slots 0, 1, 3 and 5) has a capacity of 600 bandwidth points.
Current configuration on bus mb0_mb1 has a total of 600 bandwidth points.
This configuration is within the PCI bus capacity and is supported.

PCI bus mb2 (Slots 2, 4, 6) has a capacity of 600 bandwidth points.
Current configuration on bus mb2 has a total of 0 bandwidth points
This configuration is within the PCI bus capacity and is supported.

Please refer to the following document "Cisco 7200 Series Port Adaptor
Hardware Configuration Guidelines" on Cisco.com <http://www.cisco.com>
```

FIGURE 2.7 – Vérification de la version Cisco IOS

```
R1#ping 8.8.8.8

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 8.8.8.8, timeout is 2 seconds:
!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 76/84/108 ms
R1#
```

FIGURE 2.8 – Validation de la connectivité Internet

La Figure 2.9 montre que le routeur répond correctement aux requêtes ICMP envoyées par le serveur Ubuntu.

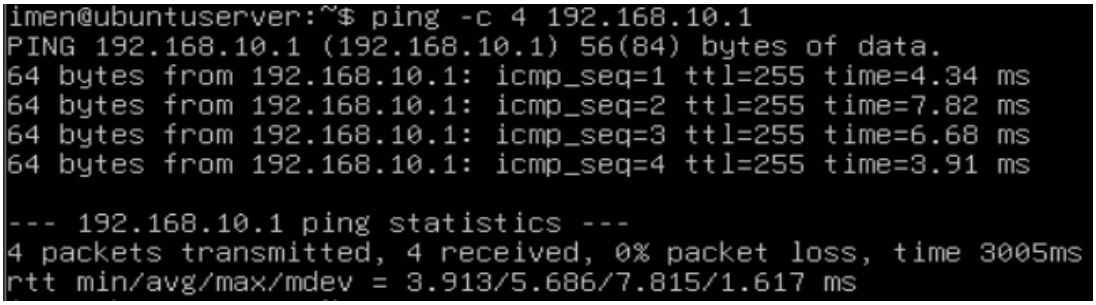


FIGURE 2.9 – Test de connectivité entre le serveur Ubuntu et le routeur Cisco

2.8.2 Communication entre Kali Linux et le routeur

Le même test a été réalisé depuis la machine Kali Linux afin de vérifier sa connectivité avec la passerelle du réseau.

Comme illustré à la Figure 2.10, les réponses ICMP confirment le bon fonctionnement de cette liaison.

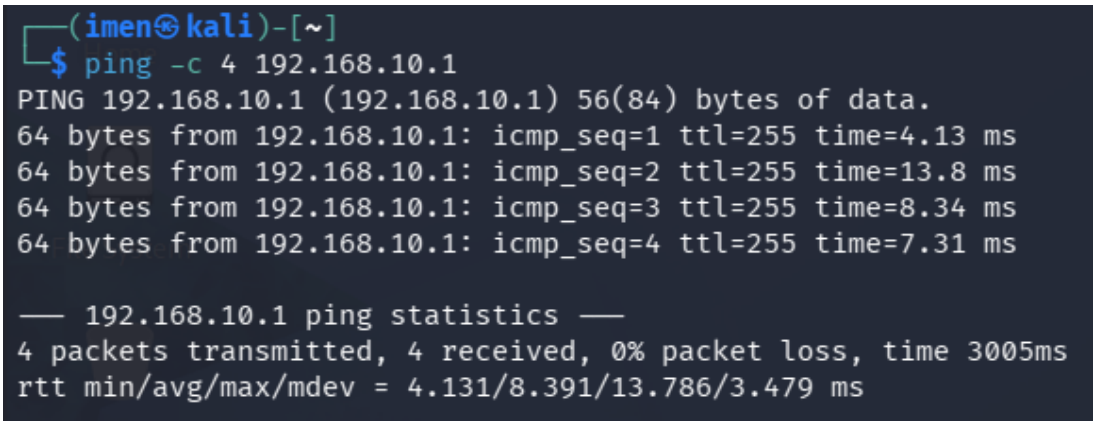


FIGURE 2.10 – Test de connectivité entre Kali Linux et le routeur Cisco

2.8.3 Communication entre le serveur Ubuntu et Kali Linux

Une communication directe entre les deux machines virtuelles a ensuite été vérifiée.

La Figure 2.11 montre que le serveur Ubuntu peut communiquer correctement avec la machine Kali Linux.

2.8.4 Communication entre Kali Linux et le serveur Ubuntu

Enfin, le test inverse a été réalisé afin de confirmer la connectivité bidirectionnelle entre les deux machines.

Les résultats présentés à la Figure 2.12 confirment que les échanges réseau sont fonctionnels dans les deux sens.

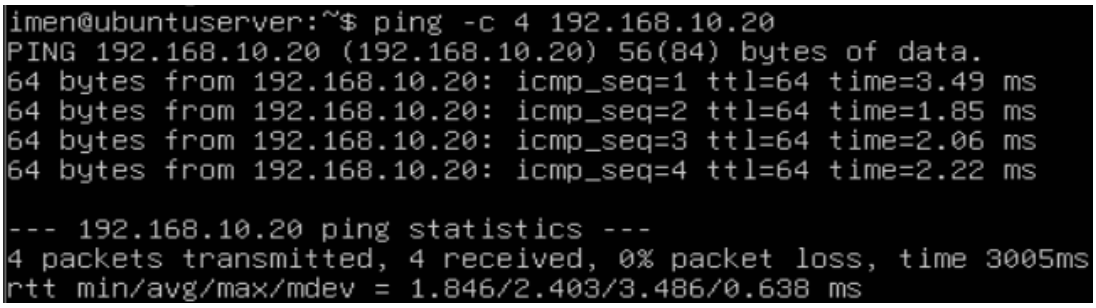


FIGURE 2.11 – Communication entre le serveur Ubuntu et Kali Linux

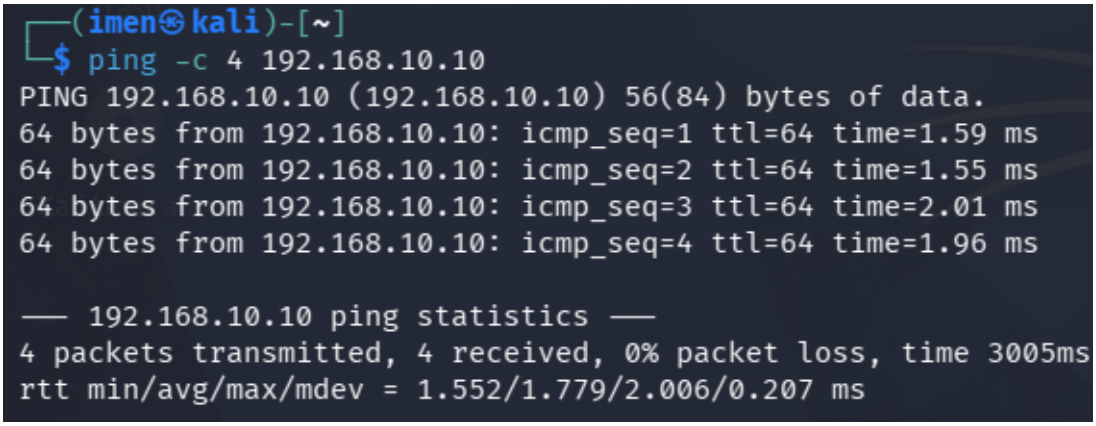


FIGURE 2.12 – Communication entre Kali Linux et le serveur Ubuntu

Les différents tests de connectivité réalisés confirment le bon fonctionnement de l'infrastructure réseau. Le routage IP, les configurations des interfaces ainsi que les communications entre les différents équipements ont été validés avec succès. Cette étape garantit que le laboratoire est prêt pour l'installation et les tests du service SSH, qui constituent la base des expérimentations présentées dans la suite du projet.

2.9 Validation du service SSH

Le protocole Secure Shell (SSH) constitue l'élément central des expérimentations réalisées dans le cadre du projet ANSDS [10]. Avant de développer les mécanismes de détection et de réponse automatique, il est indispensable de vérifier que le service SSH est correctement installé, actif et accessible depuis les autres équipements du laboratoire.

Plusieurs vérifications successives ont été réalisées afin de valider le bon fonctionnement du service.

2.9.1 Installation du serveur OpenSSH

Le serveur Ubuntu a été configuré afin d'héberger le service OpenSSH. Cette étape permet d'accepter les connexions SSH provenant de la machine Kali Linux.

La Figure 2.13 présente l'installation du serveur OpenSSH.



FIGURE 2.13 – Installation du serveur OpenSSH

2.9.2 Vérification de l'état du service

Une fois l'installation terminée, le statut du service SSH a été vérifié afin de confirmer son bon fonctionnement.

Comme l'illustre la Figure 2.14, le service est correctement démarré et prêt à accepter les connexions entrantes.

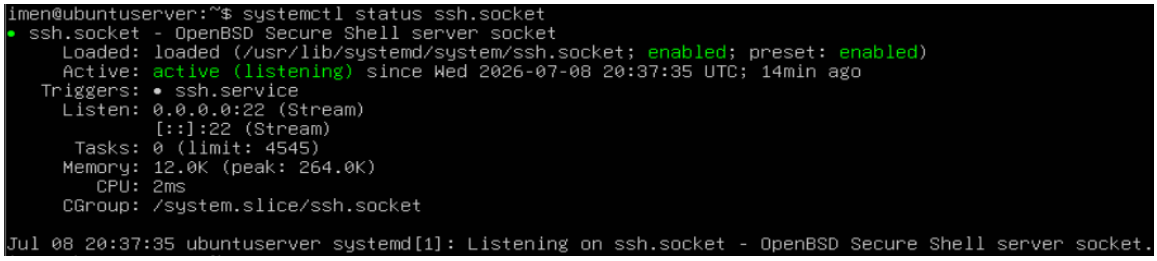


FIGURE 2.14 – Vérification de l'état du service SSH

2.9.3 Vérification du port d'écoute

Le fonctionnement du serveur SSH a également été validé en vérifiant que le port TCP 22 est en état d'écoute.

La Figure 2.15 confirme que le service écoute correctement sur le port standard utilisé par le protocole SSH [10].

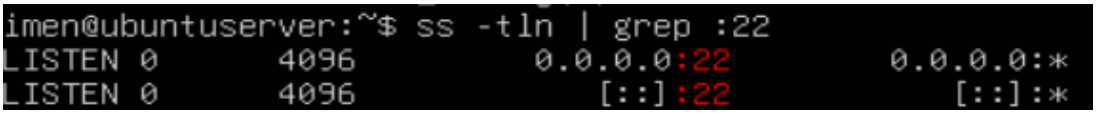


FIGURE 2.15 – Vérification de l'écoute du port TCP 22

2.9.4 Validation d'une connexion SSH

La dernière étape consiste à établir une connexion SSH entre la machine Kali Linux et le serveur Ubuntu.

La réussite de cette connexion confirme que l'ensemble des configurations réseau et du service SSH sont opérationnelles. Cette validation est essentielle, car toutes les expérimentations des chapitres suivants reposeront sur cette communication.

La Figure 2.16 présente une connexion SSH réussie entre les deux machines.

2.10 Conclusion

Au cours de ce chapitre, un laboratoire réseau complet a été conçu et configuré afin de servir de plateforme d'expérimentation pour le projet ANSDS. L'architecture mise en



FIGURE 2.16 – Connexion SSH réussie entre Kali Linux et le serveur Ubuntu

place comprend un routeur Cisco IOS, un commutateur Ethernet, un serveur Ubuntu, une machine Kali Linux ainsi qu'un accès Internet via un mécanisme NAT.

Les différentes étapes de configuration ont été validées par des tests de connectivité réseau et par la vérification du fonctionnement du service SSH. Ces résultats confirment que l'environnement expérimental est pleinement opérationnel et constitue une base fiable pour le développement des mécanismes de détection des attaques et de réponse automatique.

Le chapitre suivant sera consacré à l'analyse du trafic SSH à l'aide de Wireshark [9]. Cette étude permettra de comprendre le déroulement d'une connexion SSH, d'identifier les paquets échangés et de préparer la mise en œuvre des mécanismes de détection utilisés dans le système ANSDS.

Chapitre 3

Développement du moteur de détection SSH avec Scapy

3.1 Introduction

Après la mise en place de l'infrastructure réseau présentée dans le chapitre précédent ainsi que l'étude détaillée du protocole SSH réalisée à l'aide de Wireshark, la phase suivante du projet consiste à développer le premier composant actif du système **Automated Network Self-Defense System (ANSDS)**.

L'objectif de cette phase est de concevoir un moteur capable d'observer le trafic réseau en temps réel afin d'identifier automatiquement des comportements pouvant correspondre au début d'une attaque SSH. Contrairement aux approches reposant sur l'analyse des journaux d'authentification (*logs*), le détecteur développé dans ce projet s'appuie directement sur l'observation des paquets réseau. Cette approche permet de détecter les tentatives de connexion dès leur établissement, avant même que le serveur SSH ne traite les demandes d'authentification.

Pour réaliser cette détection, la bibliothèque Python **Scapy** [7] a été retenue. Cet outil offre des fonctionnalités avancées de capture, d'analyse et de manipulation de paquets réseau tout en permettant le développement rapide d'algorithmes de détection personnalisés. Son intégration en Python facilite également l'évolution progressive du système vers des mécanismes plus avancés tels que les notifications automatiques ou le blocage dynamique des adresses IP malveillantes.

Le développement du détecteur a été réalisé de manière incrémentale. Chaque fonctionnalité a été implémentée, testée puis validée avant de passer à l'étape suivante. Cette méthode permet de limiter les erreurs, de faciliter les phases de débogage et de garantir la fiabilité du composant développé.

Dans ce chapitre, nous présentons successivement les objectifs poursuivis, les principes de fonctionnement du détecteur, les différentes étapes de son développement, les validations expérimentales réalisées ainsi que les résultats obtenus.

3.2 Objectifs de la phase

Cette phase constitue la première étape du développement du moteur de détection d'intrusion du projet ANSDS.

Les objectifs principaux sont les suivants :

— mettre en place un environnement Python dédié au développement du détecteur ;
— installer et configurer la bibliothèque Scapy afin de permettre la capture des paquets réseau [7] ;
— observer le trafic réseau circulant sur l'interface du serveur Ubuntu ;
— filtrer uniquement le trafic correspondant au protocole SSH (port TCP 22) [10] ;
— identifier les paquets TCP SYN représentant les nouvelles tentatives de connexion [4] ;
— associer chaque tentative à son adresse IP source ;
— compter le nombre de tentatives réalisées par chaque adresse IP ;
— développer un mécanisme de détection basé sur un seuil configurable ;
— améliorer la précision de la détection en utilisant une fenêtre temporelle glissante ;
— valider expérimentalement le fonctionnement de chaque fonctionnalité développée.

À l'issue de cette phase, le système est capable d'identifier automatiquement des séries de tentatives de connexion SSH susceptibles de correspondre au début d'une attaque par force brute. Ce moteur constitue la base fonctionnelle sur laquelle seront ajoutés, dans les phases suivantes, les mécanismes de notification et de réponse automatique.

3.3 Étude du protocole SSH

Avant de développer le moteur de détection, il est indispensable de comprendre précisément le fonctionnement d'une connexion SSH ainsi que les informations qui restent observables malgré le chiffrement des échanges.

Cette étude préalable a été réalisée à l'aide de l'analyseur de protocoles Wireshark [9]. L'objectif n'était pas uniquement d'observer les paquets échangés, mais surtout d'identifier les informations réellement exploitables par le futur moteur de détection développé avec Scapy [7].

L'analyse a permis de distinguer les différentes étapes de l'établissement d'une connexion SSH, d'identifier les champs réseau accessibles sans déchiffrement et de déterminer les éléments les plus pertinents pour détecter automatiquement des comportements suspects.

3.3.1 Établissement d'une connexion SSH

Une connexion SSH repose sur le protocole TCP [10, 4]. Avant tout échange de données, une connexion fiable doit être établie entre le client et le serveur à l'aide du mécanisme appelé *Three-Way Handshake*.

Une fois cette connexion établie, le client et le serveur échangent leurs informations de version, négocient les algorithmes cryptographiques à utiliser, réalisent l'échange des clés puis activent le chiffrement de la communication [10].

À partir de cette étape, le contenu des messages devient totalement chiffré. En revanche, plusieurs informations réseau restent visibles, notamment les adresses IP, les ports TCP, les drapeaux TCP, les numéros de séquence ainsi que la fréquence des paquets échangés.

Ces informations constituent la principale source de données exploitée par le moteur de détection développé dans ce projet.

3.3.2 Analyse du Three-Way Handshake TCP

Le Three-Way Handshake représente la première étape de toute connexion SSH [4].

Cette procédure se déroule selon trois échanges successifs :

1. le client envoie un paquet TCP SYN afin de demander l'ouverture d'une nouvelle connexion [4] ;
2. le serveur répond par un paquet TCP SYN/ACK indiquant qu'il accepte la connexion ;
3. le client termine l'établissement de la connexion en envoyant un paquet TCP ACK.

La Figure 3.1 présente une vue générale d'une connexion SSH observée sous Wireshark.

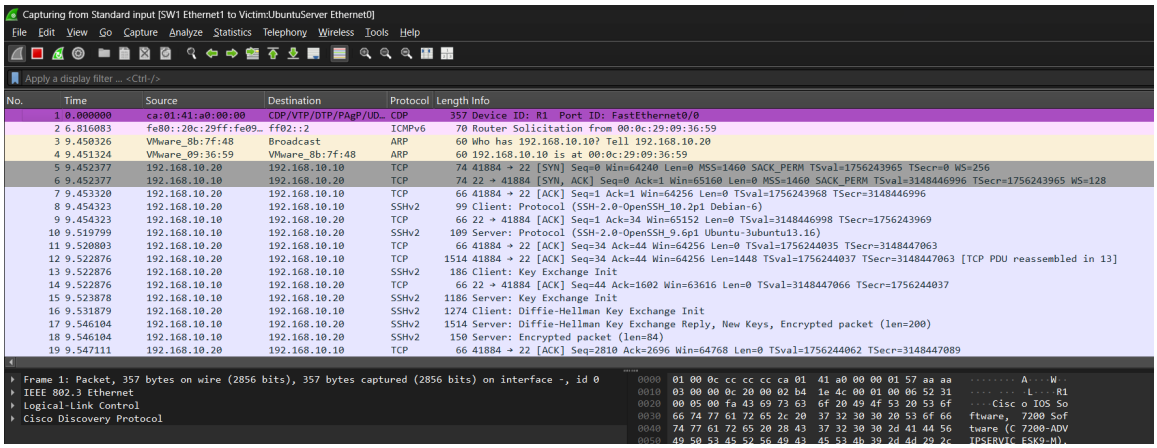


FIGURE 3.1 – Vue générale d'une connexion SSH

Les Figures 3.2, 3.3 et 3.4 présentent respectivement les trois paquets composant cette phase d'établissement.

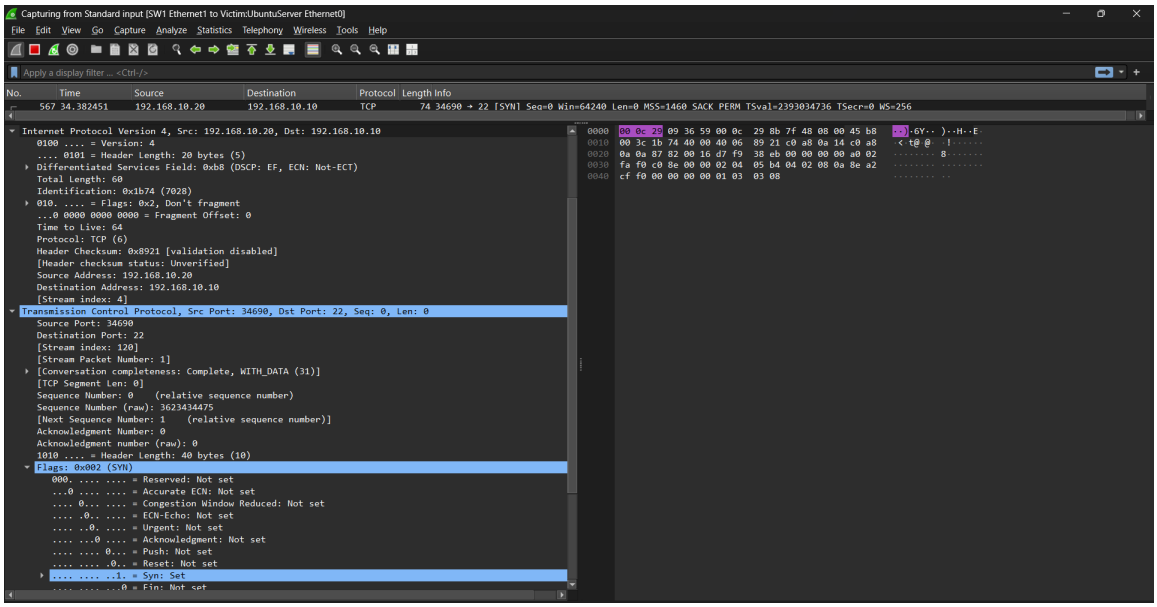


FIGURE 3.2 – Paquet TCP SYN

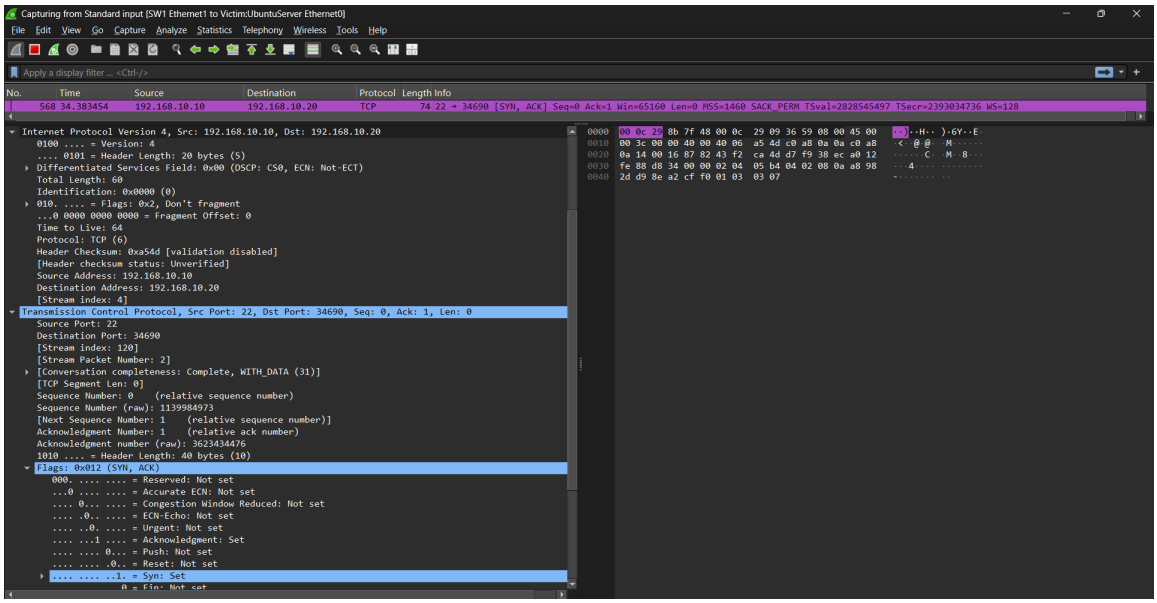


FIGURE 3.3 – Paquet TCP SYN/ACK

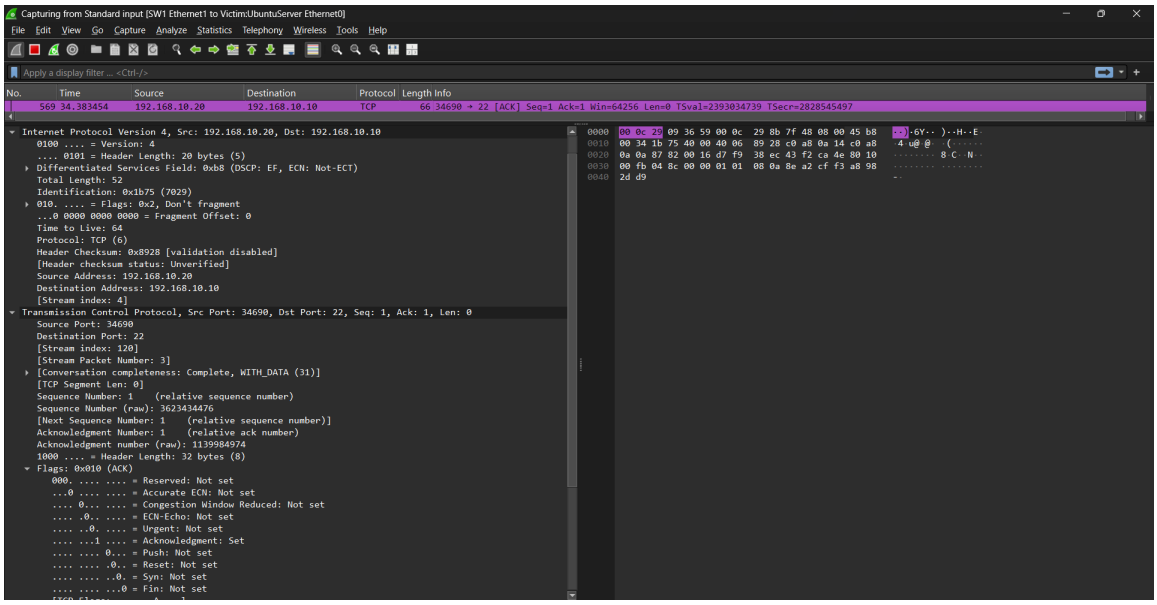


FIGURE 3.4 – Paquet TCP ACK

Parmi ces trois paquets, seul le paquet TCP SYN est utilisé par le moteur de détection. En effet, chaque tentative de connexion SSH débute obligatoirement par l'émission d'un paquet SYN vers le port TCP 22 [4, 10]. L'observation de ces paquets permet donc d'identifier rapidement les nouvelles tentatives de connexion initiées par un client.

3.3.3 Informations observables malgré le chiffrement

Après l'établissement de la connexion TCP, le protocole SSH procède à l'échange des informations de version, à la négociation des algorithmes cryptographiques puis à l'échange des clés de chiffrement [10].

Une fois cette phase terminée, les données applicatives deviennent totalement chiffrées et leur contenu n'est plus directement exploitable.

Cependant, certaines informations restent toujours visibles au niveau des couches réseau et transport :

— les adresses IP source et destination ;
— les ports TCP utilisés ;
— les drapeaux TCP (SYN, ACK, FIN, RST, etc.) [4] ;
— les numéros de séquence TCP ;
— la taille des paquets ;
— la fréquence des connexions ;
— les instants d'émission des paquets.

Ces informations sont largement suffisantes pour construire un moteur de détection basé sur l'analyse comportementale du trafic réseau.

Dans ce projet, la stratégie retenue consiste à surveiller les paquets TCP SYN destinés au port SSH, à associer chaque tentative à son adresse IP source puis à mesurer le nombre de connexions initiées pendant une fenêtre temporelle de dix secondes. Lorsque le nombre de tentatives dépasse le seuil défini, le système considère qu'un comportement anormal est détecté et génère une alerte.

3.4 Développement progressif du moteur de détection

Le développement du moteur de détection a été réalisé de manière incrémentale. Chaque nouvelle fonctionnalité a été implémentée, testée puis validée avant l'ajout de la suivante.

Cette approche présente plusieurs avantages. Elle facilite le débogage, permet de vérifier individuellement chaque composant du détecteur et garantit que les différentes fonctionnalités reposent sur une base technique stable avant leur intégration.

Le développement s'est déroulé en plusieurs étapes successives, allant de la simple capture de paquets jusqu'à la détection d'un comportement caractéristique d'une attaque par force brute sur le service SSH.

3.4.1 Première capture réseau avec Scapy

La première étape du développement consiste à vérifier le bon fonctionnement de la bibliothèque Scapy sur le serveur Ubuntu [7].

Un premier script Python est développé afin de capturer l'ensemble des paquets circulant sur l'interface réseau `ens33`. À ce stade, aucun filtrage n'est appliqué. L'objectif est uniquement de valider la capacité du programme à intercepter les paquets en temps réel.

La Figure 3.5 présente la première version du script de capture.

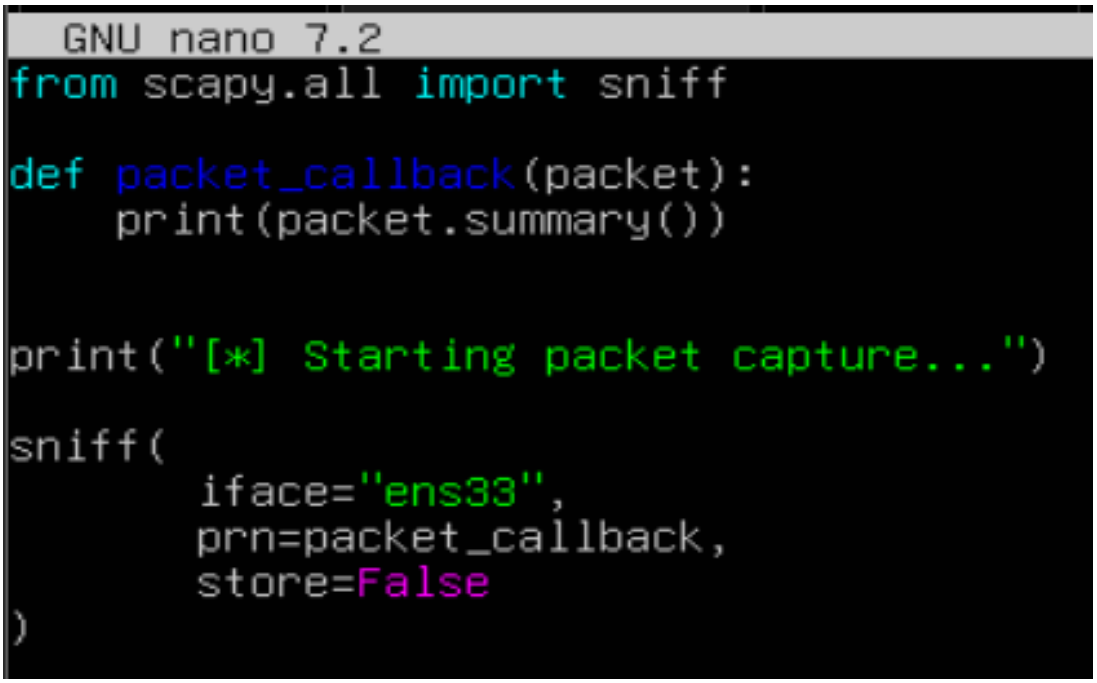


FIGURE 3.5 – Première version du détecteur développé avec Scapy

L'exécution du script confirme que Scapy capture correctement les paquets présents sur le réseau local.

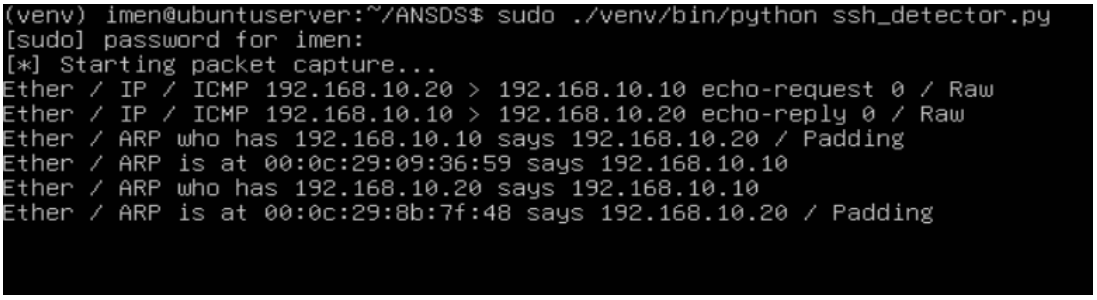


FIGURE 3.6 – Capture des paquets réseau avec Scapy

3.4.2 Filtrage du trafic SSH

La capture de l'ensemble du trafic génère un volume important d'informations ne présentant aucun intérêt pour la détection des attaques SSH.

Afin de réduire le nombre de paquets analysés, un filtre Berkeley Packet Filter (BPF) est ajouté au moteur de capture. Ce filtre limite les paquets capturés au trafic utilisant le port TCP 22.

Cette optimisation réduit considérablement la charge de traitement et concentre l'analyse uniquement sur les connexions SSH.

La Figure 3.7 illustre le résultat obtenu après l'application du filtre.

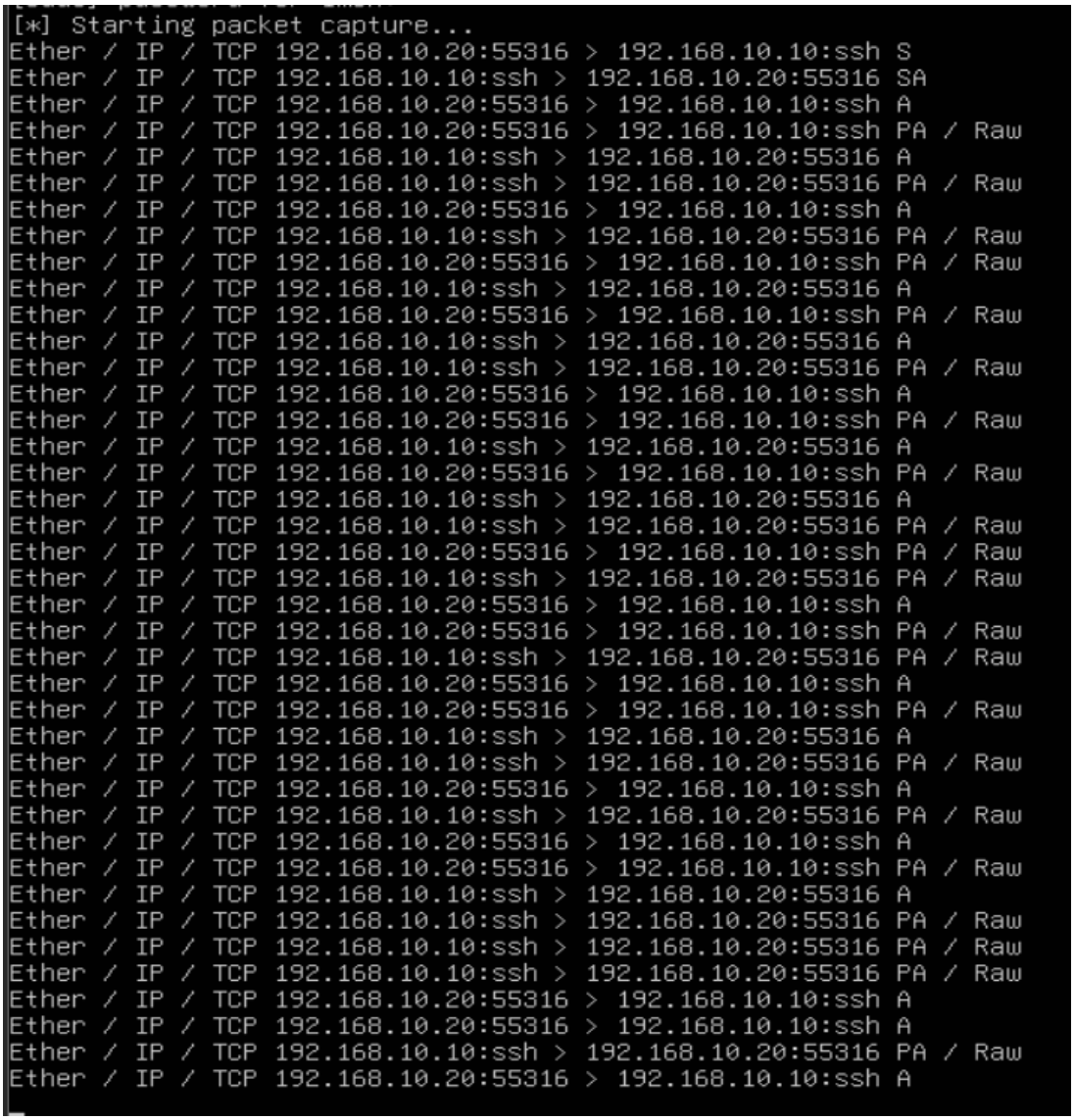


FIGURE 3.7 – Filtrage des paquets SSH

3.4.3 Détection des paquets TCP SYN

Une connexion SSH débute systématiquement par l'émission d'un paquet TCP SYN [4, 10].

Le moteur de détection est ensuite modifié afin d'identifier uniquement ces paquets. Les drapeaux TCP sont analysés afin de distinguer les nouvelles demandes de connexion des autres paquets composant la communication.

Chaque paquet SYN détecté entraîne l'affichage de l'adresse IP source et de l'adresse IP destination.

Cette étape constitue la première véritable fonctionnalité de détection développée dans le projet.

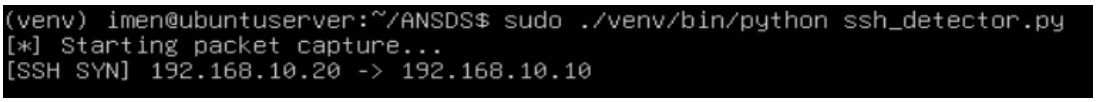


FIGURE 3.8 – Détection des paquets TCP SYN

3.4.4 Comptage des tentatives de connexion

La détection d'un unique paquet SYN ne permet pas de distinguer une connexion normale d'un comportement malveillant.

Le détecteur est donc enrichi par un mécanisme de comptage des tentatives de connexion pour chaque adresse IP source.

À chaque réception d'un paquet TCP SYN, le compteur associé à l'adresse IP émettrice est incrémenté.

Cette approche permet de suivre l'évolution du nombre de connexions initiées par chaque hôte observé sur le réseau.

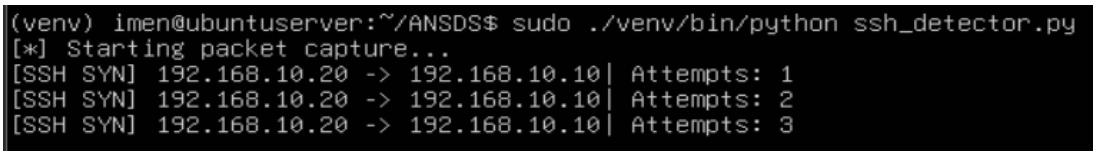


FIGURE 3.9 – Comptage des tentatives de connexion

3.4.5 Détection par seuil

Afin d'identifier automatiquement un comportement anormal, un seuil de détection est introduit.

Dans cette première version du détecteur, une alerte est générée lorsqu'une même adresse IP réalise cinq tentatives de connexion SSH.

Ce mécanisme permet de différencier une connexion isolée d'une succession rapide de tentatives pouvant correspondre au début d'une attaque par force brute.

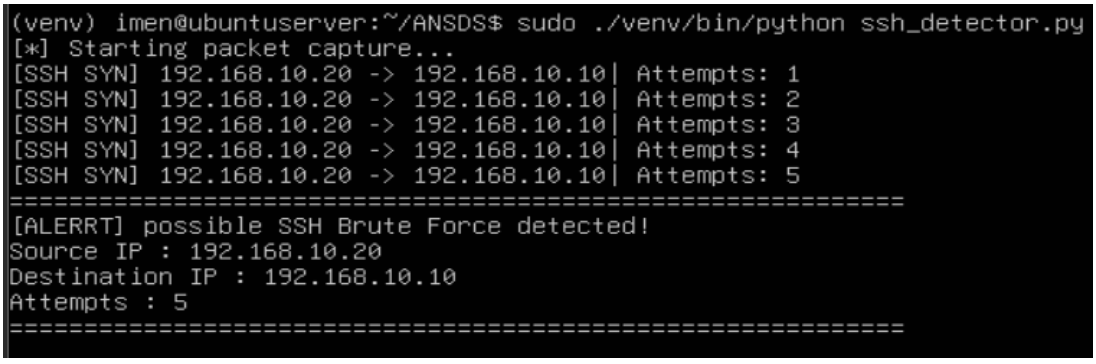


FIGURE 3.10 – Déclenchement de l'alerte après dépassement du seuil

3.4.6 Fenêtre temporelle glissante

L'utilisation d'un compteur cumulatif présente une limite importante : des tentatives espacées de plusieurs minutes peuvent conduire au déclenchement d'une alerte alors qu'elles ne correspondent pas à une attaque.

Afin d'améliorer la précision du détecteur, une fenêtre temporelle glissante de dix secondes est mise en œuvre.

Pour chaque adresse IP, le moteur mémorise l'instant de réception des paquets TCP SYN. Les événements dont l'ancienneté dépasse dix secondes sont automatiquement supprimés avant le calcul du nombre de tentatives.

Ainsi, seules les connexions récentes sont prises en compte pour la prise de décision.

Une alerte est générée uniquement lorsque cinq paquets TCP SYN ou plus sont observés dans cette fenêtre temporelle.

Cette approche réduit significativement les faux positifs et permet une détection plus représentative d'une véritable attaque par force brute.

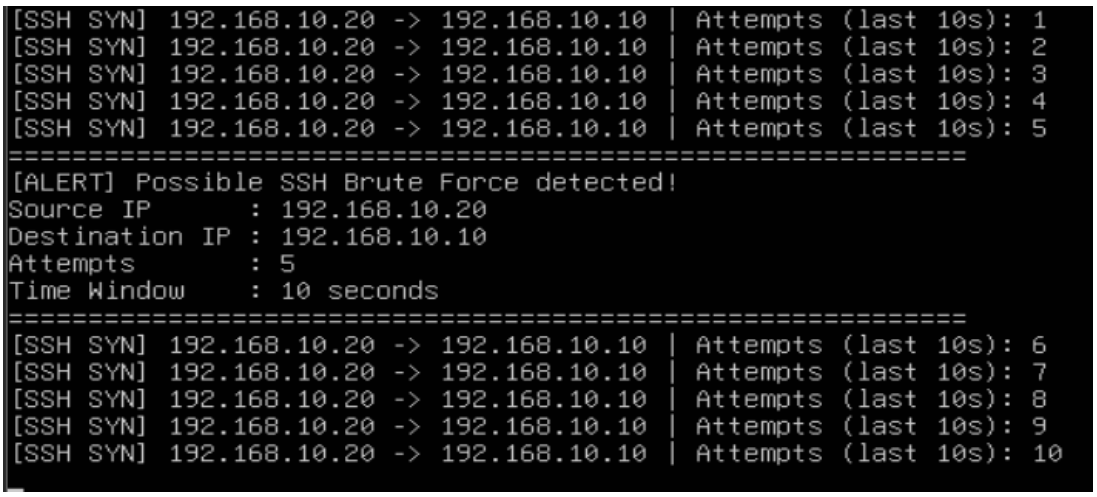


FIGURE 3.11 – Détection d'une attaque après application de la fenêtre temporelle

3.5 Validation expérimentale

Une série de tests a été réalisée afin de vérifier le bon fonctionnement du moteur de détection développé avec Scapy. Chaque fonctionnalité a été validée indépendamment avant son intégration dans le détecteur final.

La première validation a consisté à vérifier la capacité de Scapy à capturer les paquets transitant sur l'interface réseau `ens33` [7]. Après cette étape, un filtre Berkeley Packet Filter (BPF) a été appliqué afin de limiter l'observation aux seuls paquets SSH utilisant le port TCP 22.

Le détecteur a ensuite été configuré pour identifier exclusivement les paquets TCP SYN, correspondant aux nouvelles demandes d'établissement de connexion [4]. Cette fonctionnalité a permis de confirmer que chaque tentative SSH pouvait être détectée dès le début de la communication.

Une phase de validation supplémentaire a consisté à vérifier le mécanisme de comptage des tentatives de connexion par adresse IP. Plusieurs connexions successives ont été initiées depuis la machine Kali Linux afin de contrôler l'incrémentation correcte du compteur.

Enfin, des scénarios simulant une attaque par force brute ont été exécutés. Plusieurs connexions SSH ont été générées dans un intervalle inférieur à dix secondes afin de vérifier le fonctionnement du seuil de détection ainsi que de la fenêtre temporelle glissante.

L'ensemble des essais a confirmé le comportement attendu du détecteur. Les alertes sont uniquement générées lorsque le nombre de tentatives observées dépasse le seuil configuré à l'intérieur de la fenêtre temporelle définie.

3.6 Analyse des résultats

Les expérimentations réalisées montrent que l'utilisation de Scapy constitue une solution adaptée au développement d'un moteur de détection réseau léger et facilement extensible [7].

L'observation directe des paquets TCP permet de détecter les tentatives de connexion avant même que le serveur SSH ne traite les demandes d'authentification. Cette approche réduit le temps de réaction du système et facilite l'intégration de mécanismes de réponse automatique dans les phases ultérieures du projet.

L'introduction d'un filtrage BPF améliore les performances en limitant l'analyse au trafic réellement pertinent. De plus, l'utilisation d'une fenêtre temporelle glissante améliore significativement la qualité de la détection en évitant que des connexions espacées dans le temps soient interprétées comme une attaque.

Le moteur développé reste volontairement simple afin de privilégier la compréhension des mécanismes fondamentaux de détection. Cette architecture modulaire facilitera l'ajout de nouvelles fonctionnalités telles que la notification Telegram, la journalisation des événements ou encore le blocage automatique des adresses IP via un routeur Cisco.

3.7 Conclusion

Cette phase a permis de développer le premier composant opérationnel du système Automated Network Self-Defense System.

À partir d'une simple capture de paquets réseau, le détecteur a progressivement évolué vers un véritable moteur de détection capable d'identifier des comportements compatibles avec une attaque par force brute sur le service SSH.

Les différentes fonctionnalités développées comprennent la capture des paquets réseau, le filtrage du trafic SSH, la détection des paquets TCP SYN, le comptage des tentatives de connexion, la mise en œuvre d'un seuil de détection ainsi que l'utilisation d'une fenêtre temporelle glissante permettant d'améliorer la précision de l'analyse.

Les résultats obtenus valident les choix techniques retenus et démontrent la faisabilité de la solution proposée.

Le détecteur développé constitue désormais une base solide pour la suite du projet. La phase suivante sera consacrée à l'ajout des mécanismes de réponse automatique, notamment l'envoi d'alertes via Telegram et le blocage dynamique des adresses IP malveillantes sur un routeur Cisco à l'aide de la bibliothèque Netmiko.

Chapitre 4

Développement de la notification et de la réponse automatisée

4.1 Introduction

Après le développement et la validation du moteur de détection SSH présenté dans le chapitre précédent, l'étape suivante du projet Automated Network Self-Defense System (ANSDS) consiste à mettre en place les premiers mécanismes de réponse aux événements de sécurité détectés.

La simple détection d'une attaque ne constitue pas une réponse suffisante face aux menaces actuelles. Un système de sécurité efficace doit être capable de transmettre rapidement les informations relatives à l'incident afin de réduire le temps de réaction de l'administrateur, puis d'appliquer automatiquement une mesure de protection lorsque cela est nécessaire.

Dans cette phase, deux mécanismes complémentaires sont développés. Le premier est un système de notification basé sur Telegram permettant d'informer automatiquement l'administrateur lorsqu'un comportement suspect est détecté. Le second repose sur Netmiko et permet d'automatiser les interactions avec le routeur Cisco afin d'appliquer dynamiquement une ACL de blocage.

Afin de conserver une architecture logicielle modulaire, les différents composants sont développés séparément puis intégrés progressivement au moteur de détection développé avec Scapy. Cette séparation facilite la maintenance, la réutilisation du code et l'évolution future du système.

Dans ce chapitre, nous présentons successivement le développement du système de notification Telegram, son intégration avec le moteur de détection, la mise en place de l'automatisation Cisco avec Netmiko, la création et la suppression dynamique des ACL, la gestion des exceptions ainsi que les validations expérimentales réalisées.

4.2 Objectifs du chapitre

Cette phase constitue une étape importante dans l'évolution du système ANSDS, puisqu'elle permet de passer d'un système capable uniquement de détecter une activité suspecte à un système capable également de notifier l'administrateur et d'appliquer automatiquement une mesure de protection.

Les objectifs principaux sont les suivants :

— créer un bot Telegram destiné à recevoir les alertes de sécurité ;
— développer un module Python réutilisable permettant d'envoyer automatiquement des notifications ;
— intégrer ce module au moteur de détection SSH développé avec Scapy ;
— construire un message d'alerte contenant les informations essentielles relatives à l'attaque détectée ;
— transmettre automatiquement une notification dès que le seuil de détection est atteint ;
— préparer le routeur Cisco à l'automatisation distante ;
— installer et configurer la bibliothèque Netmiko ;
— développer un module Python dédié à la gestion des ACL Cisco ;
— appliquer dynamiquement une ACL permettant de bloquer une adresse IP identifiée comme malveillante ;
— supprimer automatiquement l'ACL après expiration de la durée de blocage ;
— gérer les erreurs pouvant survenir lors des communications avec le routeur ;
— intégrer l'ensemble des composants dans une chaîne de réponse automatisée ;
— valider expérimentalement le fonctionnement de chaque fonctionnalité.

4.3 Développement du système de notification Telegram

4.3.1 Présentation de Telegram Bot API

Telegram est une plateforme de messagerie instantanée proposant une interface de programmation appelée Telegram Bot API. Cette interface permet à des applications externes d'interagir automatiquement avec le service Telegram sans intervention humaine [8].

Le principe repose sur la création d'un bot, c'est-à-dire un compte spécial administré par une application. Contrairement à un utilisateur classique, un bot peut recevoir des commandes, envoyer des messages, transmettre des fichiers et communiquer automatiquement avec les utilisateurs autorisés [8].

L'accès à Telegram Bot API repose sur un jeton d'authentification (*Bot Token*) généré lors de la création du bot. Ce jeton identifie de manière unique l'application et autorise l'envoi de requêtes vers les serveurs Telegram.

Chaque conversation Telegram possède également un identifiant numérique (*Chat ID*) permettant de spécifier précisément le destinataire des notifications. L'association du Bot Token et du Chat ID permet ainsi au système de transmettre automatiquement les alertes de sécurité à l'administrateur.

Dans le cadre du projet ANSDS, Telegram est utilisé exclusivement comme canal de notification. Les informations produites par le moteur de détection sont transmises au bot afin d'informer immédiatement l'administrateur d'une activité suspecte observée sur le réseau.

4.3.2 Création du Bot Telegram

La première étape du développement consiste à créer un bot Telegram dédié aux notifications de sécurité.

Telegram met à disposition un service officiel nommé BotFather permettant de créer et d'administrer les bots. Après avoir lancé la commande `/newbot`, l'utilisateur choisit un

nom d'affichage ainsi qu'un nom d'utilisateur unique se terminant obligatoirement par le suffixe `bot`.

Une fois la création terminée, BotFather génère automatiquement le Bot Token nécessaire à l'authentification auprès de Telegram Bot API [8].

La Figure 4.1 présente la création du bot utilisé dans le cadre de ce projet.

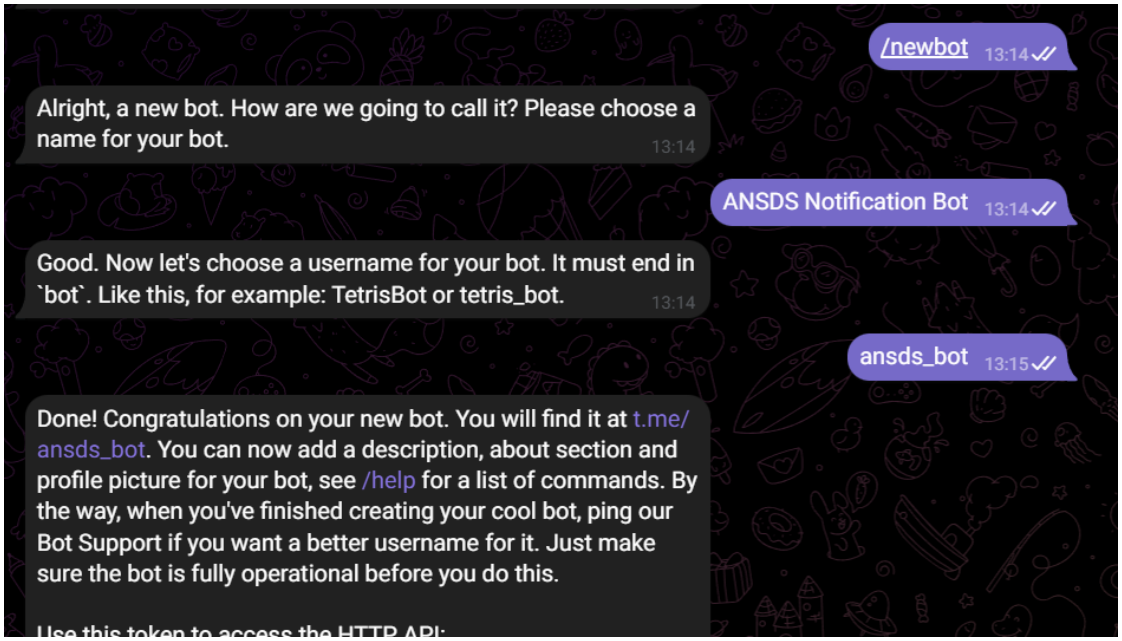


FIGURE 4.1 – Création du bot Telegram avec BotFather

Le Bot Token constitue une information sensible puisqu'il permet d'accéder au bot et d'envoyer des messages en son nom. Il doit donc être conservé de manière confidentielle et ne jamais être diffusé dans un dépôt Git public.

Après la création du bot, une première conversation privée est ouverte avec celui-ci à l'aide de la commande `/start`. Cette étape permet à Telegram d'associer un identifiant de conversation (*Chat ID*) au futur destinataire des notifications.

Le Chat ID est ensuite récupéré à l'aide de la méthode `getUpdates` proposée par Telegram Bot API [8].

La Figure 4.2 présente la récupération de cet identifiant.

Pretty-print ☑


```
{
  "ok": true,
  "result": [
    {
      "update_id": 650537957,
      "message": {
        "message_id": 2,
        "from": {
          "id": 8906361234,
          "is_bot": false,
          "first_name": "Whoami!!",
          "language_code": "fr"
        },
        "chat": {
          "id": 8906361234,
          "first_name": "Whoami!!",
          "type": "private"
        },
        "date": 1784637375,
        "text": "/start",
        "entities": [
          {
            "offset": 0,
            "length": 6,
            "type": "bot_command"
          }
        ]
      }
    }
  ]
}
```

À l'issue de cette étape, toutes les informations nécessaires à la communication avec Telegram Bot API sont disponibles. Le développement du module Python de notification peut alors commencer.

4.3.3 Développement du module de notification

Une fois les paramètres d'authentification obtenus, l'étape suivante consiste à développer un module Python capable de communiquer avec Telegram Bot API [8].

Plutôt que d'intégrer directement les appels HTTP à l'intérieur du moteur de détection, un composant indépendant nommé `telegram_notifier.py` est développé. Cette séparation des responsabilités permet de rendre le projet plus modulaire et facilite les évolutions futures.

Le module de notification est chargé exclusivement de transmettre les messages vers Telegram. Il ne réalise aucune opération de détection et ne possède aucune connaissance du fonctionnement interne du moteur Scapy. Cette approche respecte le principe de responsabilité unique (*Single Responsibility Principle*) couramment utilisé dans le développement logiciel.

La communication avec Telegram est réalisée à l'aide de la bibliothèque Python `requests`. Cette bibliothèque simplifie l'envoi de requêtes HTTP vers Telegram Bot API tout en offrant des mécanismes de gestion des erreurs permettant d'améliorer la robustesse du système.

Le module développé expose une unique fonction publique nommée `send_notification()`. Cette fonction reçoit le contenu du message à transmettre, construit la requête HTTP puis vérifie la réponse renvoyée par Telegram avant d'indiquer si l'envoi a été réalisé avec succès.

Cette architecture permet à n'importe quel futur composant du projet d'envoyer une notification sans avoir à connaître les détails de communication avec Telegram.

Une première validation est réalisée afin de vérifier le fonctionnement du module indépendamment du moteur de détection.

La Figure 4.3 présente la validation du premier message envoyé depuis Python.

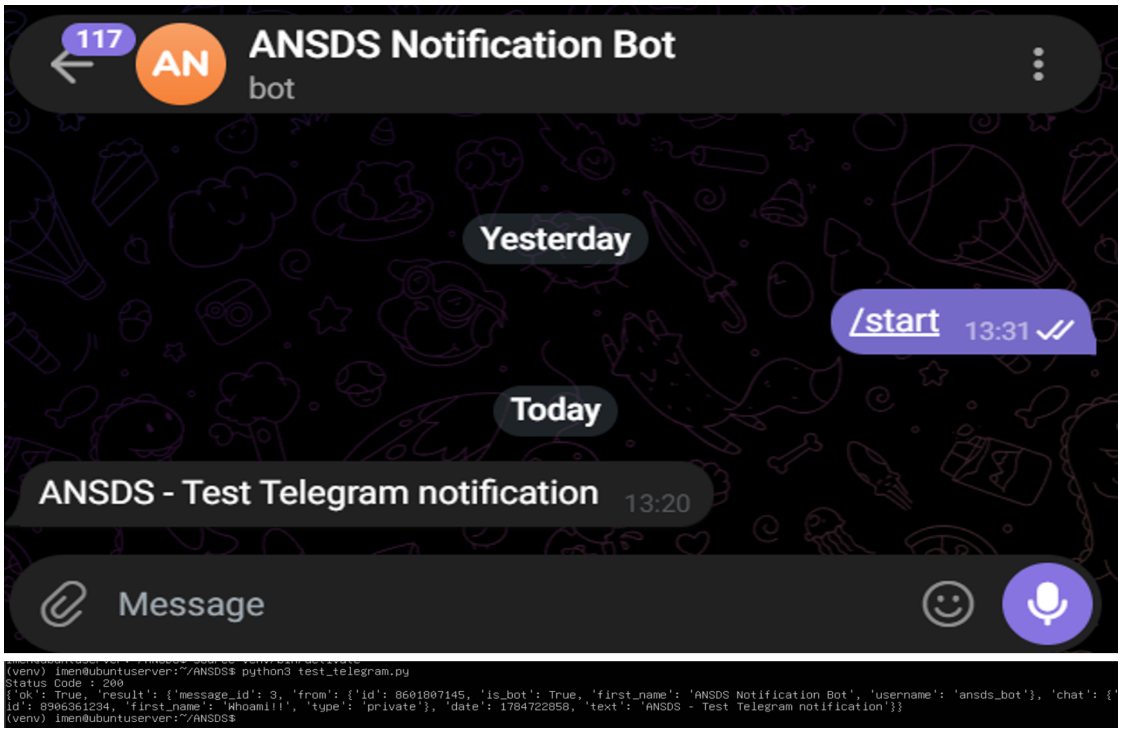


FIGURE 4.3 – Validation du premier message envoyé via Telegram Bot API

Après cette première validation, un second script est utilisé afin de vérifier que toutes les communications passent désormais exclusivement par le module `telegram_notifier.py`.

Cette étape confirme que la logique de communication est entièrement centralisée dans un composant unique, facilitant ainsi la maintenance du projet.

La Figure 4.4 présente cette validation.

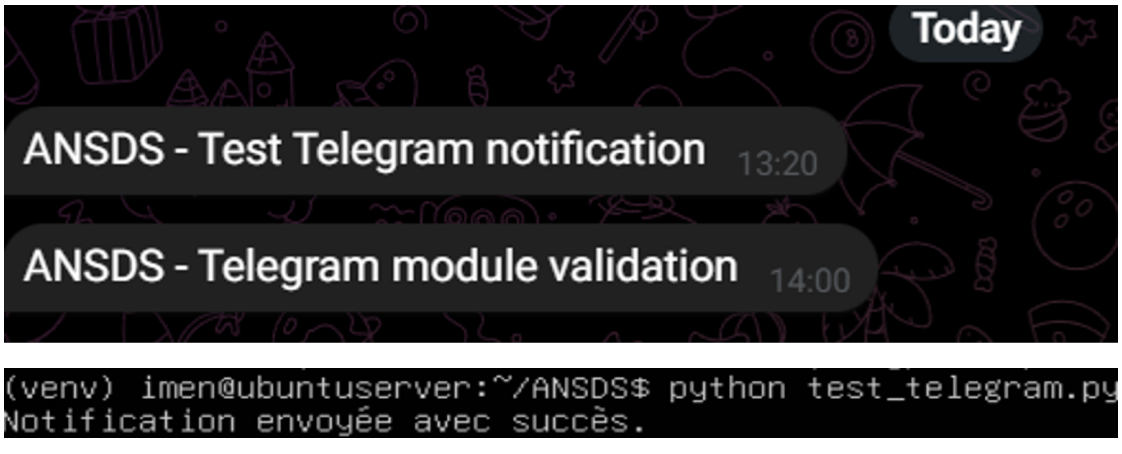


FIGURE 4.4 – Validation du module Python de notification

4.3.4 Intégration avec le moteur de détection SSH

Après la validation indépendante du module de notification, celui-ci est intégré au moteur de détection développé avec Scapy [7].

L'objectif consiste à automatiser entièrement l'envoi des alertes. Désormais, aucune intervention humaine n'est nécessaire entre la détection d'une attaque et la transmission de la notification.

Le moteur de détection continue d'assurer uniquement l'observation du trafic réseau, le comptage des tentatives de connexion et l'identification des comportements suspects. Lorsqu'une adresse IP dépasse le seuil configuré dans la fenêtre temporelle glissante, le détecteur appelle simplement la fonction `send_notification()`.

Cette approche présente plusieurs avantages. Le moteur de détection reste indépendant de Telegram, tandis que le module de notification peut évoluer sans modifier les algorithmes de détection.

Afin d'éviter la génération de notifications multiples lors d'une même attaque, le déclenchement de l'alerte est effectué uniquement lorsque le compteur atteint exactement le seuil configuré. Cette amélioration empêche l'envoi de messages répétitifs tout en garantissant qu'une seule notification est produite pour chaque nouvelle attaque détectée.

L'intégration du module est ensuite validée en générant plusieurs tentatives de connexion SSH depuis la machine Kali Linux.

Le détecteur identifie correctement les paquets TCP SYN, incrémente le compteur associé à l'adresse IP source puis déclenche automatiquement l'envoi d'une notification lorsque le seuil est atteint.

La Figure 4.5 présente le fonctionnement complet de cette intégration.

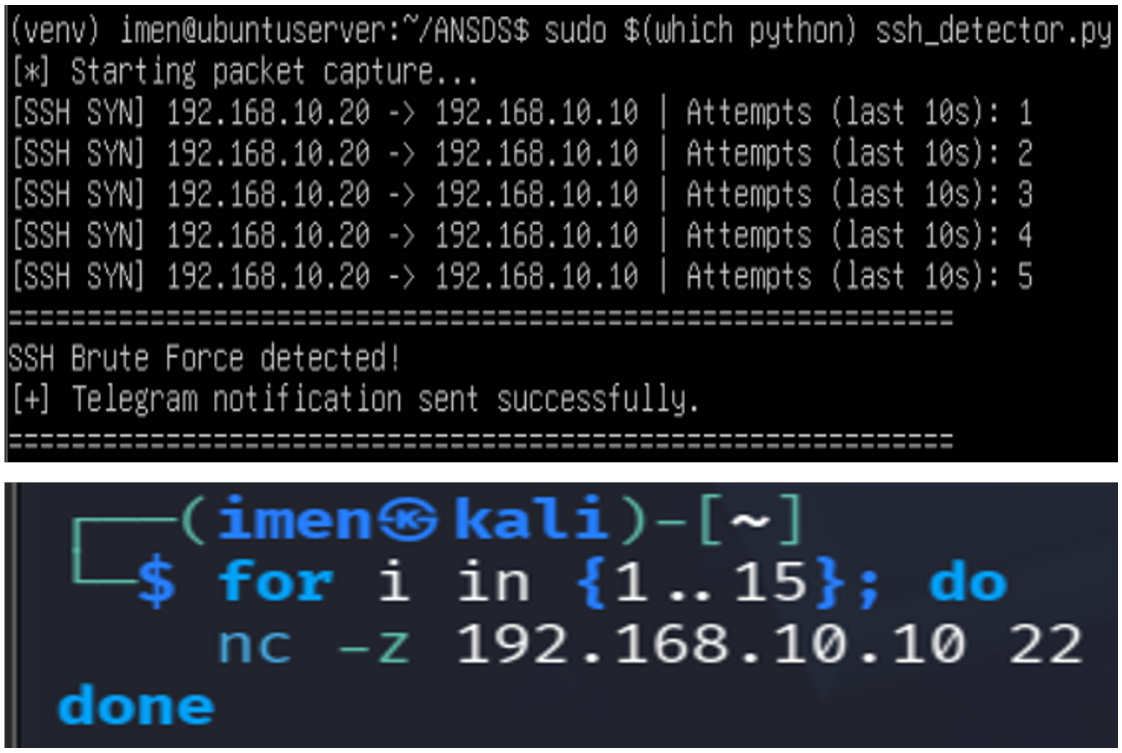


FIGURE 4.5 – Intégration du module Telegram au moteur de détection SSH

4.3.5 Construction d'une alerte professionnelle

La première version du système de notification envoyait uniquement un message générique indiquant qu'une attaque SSH avait été détectée.

Bien que cette approche permette de vérifier le bon fonctionnement de Telegram, elle ne fournit pas suffisamment d'informations pour permettre une analyse rapide de l'incident ou une automatisation des actions de réponse.

Le contenu de la notification a donc été enrichi afin de transmettre directement toutes les informations nécessaires à l'administrateur réseau.

Chaque notification contient désormais le type d'attaque détectée, le niveau de gravité, la date et l'heure de détection, les adresses IP source et destination, le port cible, le protocole utilisé, le nombre de tentatives observées ainsi que la durée de la fenêtre temporelle utilisée par le moteur de détection.

Cette structure présente également un intérêt majeur pour la suite du projet. Les informations générées par le moteur de détection pourront être directement réutilisées par le module Netmiko afin de construire automatiquement les règles ACL destinées à bloquer l'adresse IP malveillante sur le routeur Cisco.

La Figure 4.6 présente le format final de la notification générée automatiquement par le système.



FIGURE 4.6 – Notification de sécurité générée automatiquement par l'ANSDS

4.4 Validation du système de notification

4.4.1 Validation expérimentale

Une série de tests a été réalisée afin de vérifier le bon fonctionnement du système de notification développé durant cette phase. Chaque fonctionnalité a été validée individuellement avant son intégration complète au moteur de détection SSH.

La première validation a consisté à vérifier la création du bot Telegram ainsi que l'obtention des paramètres nécessaires à la communication avec Telegram Bot API [8]. Le Bot Token et le Chat ID ont été récupérés avec succès, permettant ainsi au programme Python de communiquer avec les serveurs Telegram.

Une seconde série de tests a permis de vérifier le fonctionnement du module `telegram_notifier.py`. Plusieurs messages de test ont été transmis afin de confirmer la capacité du module à envoyer automatiquement des notifications sans intervention de l'utilisateur.

Après cette validation indépendante, le module de notification a été intégré au moteur de détection développé avec Scapy [7]. Des scénarios simulant une attaque SSH ont ensuite été exécutés depuis la machine Kali Linux.

À chaque tentative de connexion, le détecteur capture les paquets TCP SYN, met à jour le compteur associé à l'adresse IP source puis compare le nombre de tentatives au seuil configuré.

Lorsque le seuil est atteint à l'intérieur de la fenêtre temporelle glissante, une unique notification est automatiquement générée et transmise à l'administrateur via Telegram.

Les différents essais réalisés confirment le bon fonctionnement de l'ensemble de la chaîne de traitement. Les notifications sont envoyées uniquement lorsqu'une activité réellement suspecte est détectée, évitant ainsi la génération d'alertes inutiles.

4.4.2 Analyse des résultats

Les expérimentations réalisées démontrent que l'intégration de Telegram Bot API constitue une solution simple, fiable et efficace pour assurer la notification automatique des incidents de sécurité [8].

La séparation entre le moteur de détection et le module de notification améliore considérablement l'architecture logicielle du projet. Chaque composant possède désormais une responsabilité clairement définie, ce qui facilite la maintenance et limite les dépendances entre les différents modules.

Le développement d'un module Python dédié permet également de centraliser l'ensemble des communications avec Telegram. Toute évolution future du système de notification pourra ainsi être réalisée sans modifier les algorithmes de détection développés avec Scapy [7].

L'amélioration apportée au mécanisme de déclenchement des alertes constitue également un résultat important. Le remplacement d'une condition de type `count >= THRESHOLD` par une comparaison stricte `count == THRESHOLD` permet d'éviter l'envoi de notifications multiples lors d'une même attaque tout en conservant une bonne réactivité.

L'enrichissement du contenu des notifications représente une étape essentielle dans la construction du système d'auto-défense réseau. Les informations transmises ne servent plus uniquement à informer l'administrateur, mais constituent également les données qui seront exploitées par le module de réponse automatique.

Cette conception permet de préparer naturellement l'intégration de Netmiko sans modifier l'architecture existante. Les informations relatives à l'attaque pourront être direc-

tement utilisées pour générer dynamiquement les règles de filtrage sur le routeur Cisco.

4.5 Développement du mécanisme de réponse automatique

Après la mise en place du système de notification, l'étape suivante consiste à automatiser la réponse réseau. L'objectif est de permettre au système ANSDS de communiquer automatiquement avec le routeur Cisco et d'appliquer une mesure de protection lorsqu'une adresse IP malveillante est identifiée.

4.5.1 Préparation du routeur Cisco

Avant d'automatiser les opérations de configuration, le routeur Cisco doit être préparé afin d'autoriser une connexion distante sécurisée depuis le serveur Ubuntu.

Cette préparation constitue une étape indispensable pour permettre au module Netmiko d'établir une session SSH avec le routeur et d'exécuter les commandes nécessaires à la gestion des ACL [1, 2].

Configuration SSH du routeur

La configuration du service SSH est réalisée sur le routeur Cisco afin d'autoriser les connexions distantes authentifiées [10, 2].

Les paramètres nécessaires sont configurés puis vérifiés avant de procéder à l'intégration avec Netmiko.

Validation de la connexion SSH

Une première connexion SSH est réalisée manuellement afin de vérifier que le routeur accepte correctement les connexions provenant du serveur Ubuntu.

Cette validation permet de distinguer les éventuels problèmes liés à la configuration réseau ou au service SSH des problèmes qui pourraient être introduits ultérieurement par Netmiko.

4.5.2 Installation et configuration de Netmiko

Netmiko est une bibliothèque Python spécialisée dans l'automatisation des équipements réseau. Elle permet notamment d'établir des connexions SSH avec des équipements Cisco et d'exécuter automatiquement des commandes de configuration [1].

Dans le cadre du projet ANSDS, Netmiko constitue l'interface entre le moteur logiciel développé en Python et le routeur Cisco.

Installation de la bibliothèque

La bibliothèque Netmiko est installée dans l'environnement Python utilisé pour le projet.

Une fois l'installation terminée, une vérification est effectuée afin de s'assurer que la bibliothèque peut être importée correctement par les scripts Python.

Premier test de connexion

Un premier script est développé afin de vérifier la capacité de Netmiko à établir une session SSH avec le routeur Cisco [1].

Cette étape permet de valider les paramètres de connexion, notamment l'adresse IP du routeur, le nom d'utilisateur, le mot de passe ainsi que le type d'équipement Cisco.

Récupération automatique des informations du routeur

Après validation de la connexion, le script est utilisé pour exécuter automatiquement des commandes de consultation sur le routeur.

Cette fonctionnalité confirme que le programme Python dispose des droits nécessaires pour interagir avec l'équipement et constitue la base de la future automatisation des ACL.

4.5.3 Développement du module Cisco

Afin de conserver une architecture modulaire, les opérations relatives au routeur Cisco sont regroupées dans un module Python indépendant nommé `cisco_acl_manager.py`.

Architecture modulaire

Le module Cisco est séparé du moteur de détection et du module Telegram. Cette organisation permet de conserver une séparation claire des responsabilités :

- Scapy assure la capture et l'analyse du trafic [7] ;
- le moteur de détection identifie les comportements suspects ;
- Telegram assure la notification ;
- Netmiko assure la communication avec le routeur [1] ;
- le module Cisco assure la gestion des ACL [2].

Cette architecture facilite la maintenance du système et permet de faire évoluer chaque composant indépendamment.

Création dynamique d'une ACL

Lorsqu'une adresse IP malveillante est identifiée, le module Cisco peut générer dynamiquement une règle `ACL` correspondant à cette adresse [2].

L'adresse IP détectée par le moteur de sécurité est transmise au module Cisco, qui construit ensuite la commande nécessaire à l'application de la règle sur le routeur.

Cette approche permet d'éviter la configuration manuelle des adresses bloquées et constitue le principe central de la réponse automatique du système.

Suppression automatique d'une ACL

Le mécanisme de blocage est conçu pour être temporaire. Après expiration de la durée configurée, le module Cisco supprime automatiquement la règle précédemment créée.

Cette fonctionnalité évite qu'une ACL temporaire reste active indéfiniment et permet au réseau de revenir automatiquement à son état initial.

4.5.4 Gestion des exceptions

L'automatisation d'un équipement réseau nécessite également une gestion appropriée des erreurs susceptibles de survenir lors de la communication.

Gestion des erreurs de communication

Les erreurs de connexion, les problèmes d'authentification ou les indisponibilités temporaires du routeur peuvent empêcher l'exécution correcte d'une commande.

Le module Cisco doit donc intercepter ces exceptions afin d'éviter l'arrêt brutal du programme principal.

Utilisation des blocs `try`, `except` et `finally`

Les blocs `try`, `except` et `finally` permettent de séparer l'exécution normale du traitement des erreurs et des opérations devant être réalisées systématiquement lors de la fermeture de la connexion.

Cette organisation améliore la robustesse du système et garantit que les ressources utilisées lors de la communication avec le routeur sont correctement libérées.

Validation du mécanisme de gestion des exceptions

Des tests sont réalisés afin de vérifier le comportement du module en cas d'erreur de communication avec le routeur.

La validation confirme que les exceptions sont correctement interceptées et que le moteur principal peut continuer son fonctionnement sans interruption imprévue.

4.5.5 Intégration avec le moteur de détection

Après validation indépendante du module Cisco, celui-ci est intégré au moteur de détection.

Transmission de l'adresse IP détectée

Lorsqu'un comportement suspect est confirmé, l'adresse IP source identifiée par le moteur de détection est transmise au module Cisco.

Cette information constitue l'élément principal utilisé pour construire dynamiquement la règle ACL de blocage.

Enchaînement automatique des modules

L'intégration permet désormais d'obtenir une première chaîne automatisée :

Détection SSH → Alerte Telegram → Netmiko → ACL Cisco

Le moteur de détection identifie l'événement, Telegram informe l'administrateur et Netmiko applique automatiquement la mesure de protection sur le routeur Cisco.

Architecture finale du système

À l'issue de cette phase, les différents composants développés sont interconnectés tout en conservant leur séparation fonctionnelle.

Le moteur de détection est chargé de l'analyse du trafic, le module Telegram assure la notification et le module Cisco réalise la réponse réseau.

Cette architecture constitue la base de l'évolution future vers une détection multi-indicateurs et une réponse automatisée plus avancée.

4.5.6 Validation expérimentale

Une série de tests est réalisée afin de valider progressivement l'ensemble du mécanisme de réponse automatique.

Les validations portent notamment sur :

— la connexion SSH au routeur Cisco ;
— l'installation et le fonctionnement de Netmiko ;
— l'exécution automatique de commandes ;
— la création dynamique d'une ACL ;
— le blocage de l'adresse IP détectée ;
— la suppression automatique de l'ACL ;
— la gestion des erreurs de communication ;
— l'intégration avec le moteur de détection ;
— l'enchaînement entre la détection, la notification et la réponse.

Les différents essais permettent de vérifier que l'adresse IP détectée peut être transmise automatiquement au module Cisco et utilisée pour appliquer la règle de blocage correspondante.

4.6 Analyse des résultats

Les expérimentations réalisées montrent que l'intégration de Telegram et de Netmiko permet de transformer progressivement le moteur de détection en un système capable non seulement d'identifier une activité suspecte, mais également d'engager une réponse automatisée.

La séparation des responsabilités entre les différents modules constitue un résultat important de cette phase. Le moteur de détection reste consacré à l'analyse du trafic, tandis que les modules Telegram et Cisco assurent respectivement la notification et la réponse réseau.

L'utilisation de Netmiko permet également de supprimer la nécessité d'intervenir manuellement sur le routeur Cisco après chaque détection. L'adresse IP identifiée par le moteur peut être transmise directement au module Cisco afin de générer la règle de filtrage correspondante.

La suppression automatique de l'ACL constitue également une fonctionnalité importante. Le blocage n'est pas permanent et le réseau peut revenir automatiquement à son état initial après expiration de la durée configurée.

La chaîne de traitement obtenue à l'issue de cette phase peut ainsi être résumée comme suit :

Détection → Notification → Réponse automatique → Blocage temporaire → Suppression de l'ACL

Cette architecture prépare également l'évolution du système vers une détection multi-indicateurs, qui sera développée dans la phase suivante.

4.7 Conclusion

Cette phase a permis de faire évoluer le système ANSDS d'un moteur de détection capable d'identifier des tentatives SSH vers une architecture intégrant les premiers mécanismes de notification et de réponse automatique.

Le système est désormais capable de détecter un comportement suspect, d'informer automatiquement l'administrateur via Telegram et d'interagir avec un routeur Cisco à l'aide de Netmiko.

L'intégration du module Cisco permet d'appliquer dynamiquement une ACL destinée à bloquer l'adresse IP identifiée. Le caractère temporaire du blocage ainsi que la suppression automatique de l'ACL permettent de compléter le cycle de protection sans intervention manuelle.

La séparation des responsabilités entre Scapy, le moteur de détection, Telegram, Netmiko et le module de gestion Cisco fournit une architecture modulaire et évolutive.

Cette phase constitue ainsi une étape essentielle vers l'objectif final d'un système d'auto-défense réseau capable de détecter plusieurs catégories de comportements malveillants, de corréler les événements et d'appliquer automatiquement une réponse adaptée.

Chapitre 5

Intégration et automatisation de la réponse sur le routeur Cisco

5.1 Introduction

Après le développement du moteur de détection des attaques SSH présenté dans le chapitre précédent ainsi que l'intégration des notifications automatiques via Telegram, l'étape suivante du projet consiste à mettre en œuvre un mécanisme de réponse automatique capable de neutraliser immédiatement une source d'attaque détectée.

L'objectif de cette phase est de connecter le système *Automated Network Self-Defense System (ANSDS)* à un routeur Cisco IOS afin d'automatiser l'application de contre-mesures réseau. Contrairement à une approche reposant uniquement sur la génération d'alertes, le système développé est désormais capable d'agir directement sur l'infrastructure réseau en créant dynamiquement une liste de contrôle d'accès (*Access Control List* – ACL) destinée à bloquer l'adresse IP de l'attaquant.

Pour réaliser cette automatisation, la bibliothèque Python *Netmiko* a été retenue [1]. Cette bibliothèque permet d'établir des connexions SSH sécurisées avec des équipements réseau tout en simplifiant l'exécution de commandes Cisco IOS [1]. Son intégration facilite le développement d'un module réutilisable capable d'administrer automatiquement le routeur sans intervention humaine.

Le développement a été réalisé de manière progressive afin de garantir la fiabilité de chaque composant. Les différentes étapes comprennent la préparation du routeur Cisco pour les connexions SSH, la validation des communications entre le serveur Ubuntu et le routeur, le développement du module Cisco, l'ajout d'une gestion robuste des exceptions, puis l'intégration complète avec le moteur de détection développé lors de la phase précédente.

À l'issue de cette phase, le système ANSDS est capable de détecter une attaque SSH, de transmettre une notification via Telegram, puis d'établir automatiquement une connexion SSH avec le routeur Cisco afin de créer une ACL bloquant immédiatement l'adresse IP malveillante.

Dans ce chapitre, nous présentons successivement les objectifs poursuivis, la préparation du routeur Cisco, l'installation et la configuration de Netmiko, le développement du module de réponse automatique, les mécanismes de gestion des exceptions, l'intégration avec le moteur de détection ainsi que les validations expérimentales réalisées.

5.2 Objectifs de la phase

Cette phase constitue la première étape du développement du mécanisme de réponse automatique du projet ANSDS.

Les objectifs principaux sont les suivants :

— préparer le routeur Cisco afin d'accepter les connexions SSH sécurisées ;
— valider la communication SSH entre le serveur Ubuntu et le routeur Cisco ;
— installer et configurer la bibliothèque Python Netmiko ;
— développer un premier script permettant d'établir une connexion distante avec le routeur ;
— récupérer automatiquement les principales informations du routeur, notamment le nom d'hôte, la version du système Cisco IOS et les interfaces réseau ;
— concevoir un module Cisco entièrement modulaire et réutilisable ;
— développer une fonction capable de créer dynamiquement une liste de contrôle d'accès (ACL) afin de bloquer l'adresse IP d'un attaquant détecté ;
— développer une fonction permettant de supprimer automatiquement cette ACL lorsque cela est nécessaire ;
— protéger l'ensemble des opérations Netmiko à l'aide d'une gestion complète des exceptions afin d'assurer la robustesse du système en cas d'échec de connexion ou d'erreur d'authentification ;
— intégrer le module Cisco au moteur de détection développé lors de la phase précédente afin d'automatiser entièrement la chaîne de détection et de réponse ;
— valider expérimentalement le fonctionnement complet du système de réponse automatique.

À l'issue de cette phase, le système ANSDS est capable d'établir automatiquement une connexion SSH avec un routeur Cisco, d'y créer dynamiquement une ACL afin de bloquer une adresse IP malveillante détectée par Scapy, puis de poursuivre sa surveillance du trafic réseau sans interruption. Cette phase constitue une étape essentielle vers la mise en œuvre d'un système complet de détection et de réponse automatique aux attaques réseau.

5.3 Préparation du routeur Cisco

Avant de développer le mécanisme de réponse automatique, il a été nécessaire de préparer le routeur Cisco afin qu'il puisse accepter des connexions SSH provenant du serveur Ubuntu.

Cette étape constitue un prérequis indispensable au fonctionnement du module développé avec Netmiko. En effet, cette bibliothèque s'appuie sur le protocole SSH pour établir une connexion distante avec les équipements réseau et exécuter automatiquement des commandes Cisco IOS [1, 10, 2].

La préparation du routeur comprend la configuration du service SSH ainsi que la validation d'une première connexion distante depuis le serveur Ubuntu. Ces vérifications permettent de garantir que l'environnement est correctement configuré avant de commencer le développement du module de réponse automatique.

5.3.1 Configuration SSH du routeur

Le routeur Cisco a été configuré afin d'autoriser les connexions SSH sécurisées [10, 2].

Cette configuration comprend notamment la définition du nom de domaine, la création d'un utilisateur local, la génération d'une paire de clés RSA ainsi que l'activation du protocole SSH version 2 [2].

Ces paramètres permettent au routeur d'accepter des connexions d'administration distantes tout en bénéficiant des mécanismes de sécurité offerts par le protocole SSH [10, 2].

La Figure 5.1 présente une connexion SSH réussie entre le serveur Ubuntu et le routeur Cisco.

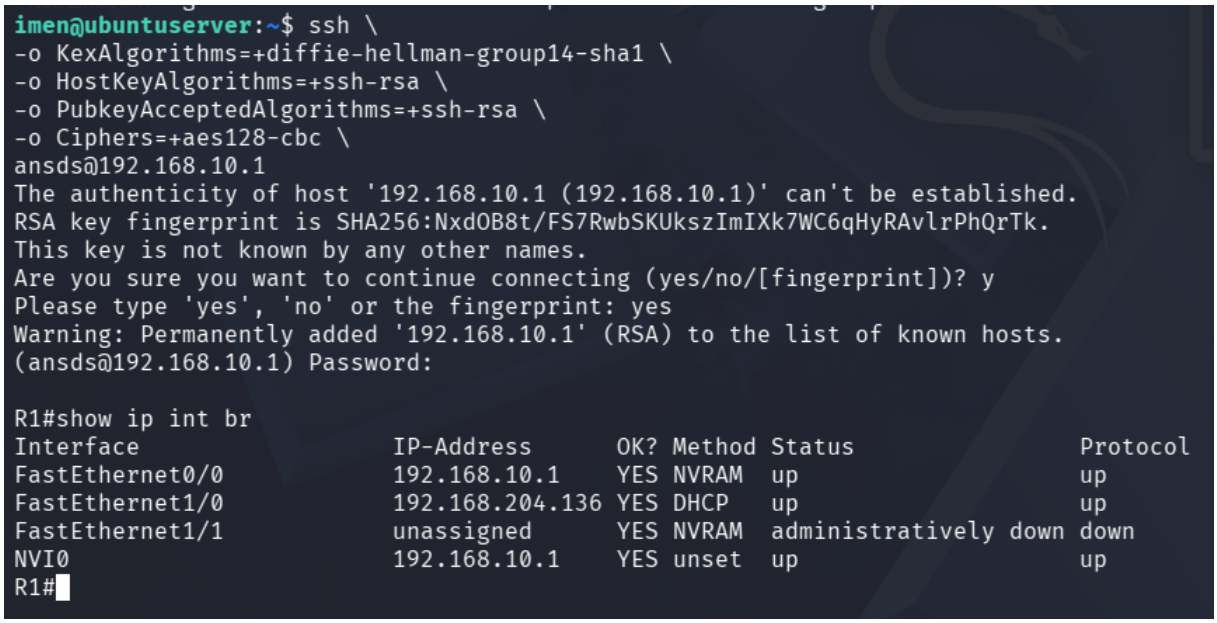


FIGURE 5.1 – Connexion SSH manuelle entre le serveur Ubuntu et le routeur Cisco

5.3.2 Validation de la connexion SSH

Une fois la configuration terminée, une connexion SSH a été établie avec succès entre le serveur Ubuntu et le routeur Cisco.

Cette validation confirme le bon fonctionnement de la configuration SSH ainsi que la connectivité réseau entre les deux équipements.

Elle garantit également que le routeur est prêt à être administré automatiquement par le futur module Cisco développé avec Netmiko.

Cette étape est essentielle, car l'ensemble des opérations de réponse automatique réalisées dans les sections suivantes reposent sur cette connexion SSH.

5.4 Installation et configuration de Netmiko

Après avoir validé la communication SSH entre le serveur Ubuntu et le routeur Cisco, l'étape suivante consiste à intégrer une bibliothèque Python capable d'automatiser les interactions avec les équipements réseau.

Dans le cadre du projet ANSDS, la bibliothèque *Netmiko* a été retenue en raison de sa simplicité d'utilisation et de sa compatibilité avec les équipements Cisco IOS. Cette bibliothèque permet d'établir des connexions SSH, d'exécuter des commandes de supervision ou de configuration et de récupérer automatiquement les informations retournées

par le routeur [1].

Avant de développer le module de réponse automatique, plusieurs validations ont été réalisées afin de vérifier le bon fonctionnement de Netmiko ainsi que la communication entre Python et le routeur Cisco.

5.4.1 Installation de la bibliothèque

La bibliothèque Netmiko a été installée dans l'environnement virtuel Python utilisé durant le développement du projet.

L'utilisation d'un environnement virtuel permet d'isoler les dépendances logicielles du projet, de garantir la reproductibilité de l'environnement de développement et de faciliter son déploiement sur une autre machine.

Après l'installation, plusieurs vérifications ont été effectuées afin de confirmer que la bibliothèque était correctement installée et prête à être utilisée.

La Figure 5.2 présente l'installation et la validation de Netmiko.

```
(venv) imen@ubuntuserver:~/ANSDS$ pip show netmiko
Name: netmiko
Version: 4.7.0
Summary: Multi-vendor library to simplify legacy CLI connections to network devices
Home-page: https://github.com/ktbyers/netmiko
Author: Kirk Byers
Author-email: ktbyers@twb-tech.com
License: MIT
Location: /home/imen/ANSDS/venv/lib/python3.12/site-packages
Requires: ntc-templates, paramiko, pyserial, pyyaml, rich, ruamel.yaml, scp, textfsm
Required-by:
(venv) imen@ubuntuserver:~/ANSDS$ python
Python 3.12.3 (main, Jun 19 2026, 12:46:00) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from netmiko import ConnectHandler
>>> print("Netmiko installé avec succès")
Netmiko installé avec succès
>>>
```

FIGURE 5.2 – Installation de la bibliothèque Netmiko

5.4.2 Premier test de connexion

Une fois Netmiko installé, un premier script de validation a été développé afin de vérifier la capacité de Python à établir une connexion SSH avec le routeur Cisco.

Ce script utilise la fonction `ConnectHandler()` de Netmiko afin d'ouvrir une session SSH en utilisant les paramètres de connexion du routeur [1]. L'objectif de cette première expérimentation est de confirmer que les échanges entre le serveur Ubuntu et le routeur Cisco peuvent être automatisés sans intervention manuelle.

La réussite de cette connexion constitue une étape essentielle avant le développement des fonctions de configuration automatique qui seront intégrées au système ANSDS.

La Figure 5.3 présente le résultat de cette première connexion.

```
(venv) imen@ubuntuserver:~/ANSDS$ python test_netmiko.py
Connexion établie avec succès.
```

FIGURE 5.3 – Première connexion établie avec Netmiko

5.4.3 Récupération automatique des informations du routeur

Après la validation de la connexion SSH, le script de test a été enrichi afin de récupérer automatiquement plusieurs informations relatives au routeur Cisco.

Les commandes exécutées permettent notamment d'obtenir le nom d'hôte du routeur, la version du système Cisco IOS ainsi que la liste des interfaces réseau configurées.

Cette étape démontre que Netmiko ne se limite pas à l'établissement d'une connexion SSH, mais permet également d'automatiser l'exécution de commandes d'administration sur les équipements Cisco [1]. Cette capacité constitue la base des futures opérations de configuration réalisées par le module de réponse automatique.

La Figure 5.4 présente les informations récupérées automatiquement.

```
(venv) imen@ubuntuserver:~/ANSDS$ python test_netmiko.py
============================================================
Hostname
============================================================
hostname R1

============================================================
IOS Version
============================================================
Cisco IOS Software, 7200 Software (C7200-ADVIPSERVICESK9-M), Version 15.0(1)M, RELEASE SOFTWARE (fc2)

============================================================
Interfaces
============================================================
Interface                  IP-Address      OK? Method Status                Protocol
FastEthernet0/0            192.168.10.1    YES NVRAM  up                    up
FastEthernet1/0            192.168.204.136 YES DHCP   up                    up
FastEthernet1/1            unassigned      YES NVRAM  administratively down down
NVI0                       192.168.10.1    YES unset  up                    up
```

FIGURE 5.4 – Récupération automatique des informations du routeur Cisco

5.5 Développement du module Cisco

Après la validation de la communication entre Python et le routeur Cisco, l'étape suivante consiste à développer un module dédié à la réponse automatique du système ANSDS.

Afin de garantir une architecture modulaire et facilement maintenable, toutes les fonctionnalités liées au routeur Cisco ont été regroupées dans un module indépendant nommé `cisco_acl_manager.py`. Cette approche permet de séparer les opérations de détection des opérations de réponse automatique tout en facilitant la réutilisation du code dans les phases suivantes du projet.

Le module développé est responsable de l'établissement de la connexion SSH avec le routeur Cisco, de l'exécution des commandes de configuration, de la création et de la suppression des listes de contrôle d'accès (ACL), ainsi que de la gestion des erreurs pouvant survenir lors des communications avec l'équipement réseau [1, 2].

5.5.1 Architecture modulaire

Le projet ANSDS repose sur une architecture modulaire dans laquelle chaque composant possède une responsabilité bien définie.

Le moteur de détection développé avec Scapy est chargé d'observer le trafic réseau et d'identifier les comportements suspects [7]. Le module Telegram assure l'envoi des notifications tandis que le module Cisco est responsable de la réponse automatique sur l'infrastructure réseau.

Cette séparation des responsabilités améliore la lisibilité du code, facilite les opérations de maintenance et permet de faire évoluer indépendamment chacun des modules du système.

5.5.2 Création du module `cisco_acl_manager.py`

Le module `cisco_acl_manager.py` constitue le cœur du mécanisme de réponse automatique développé durant cette phase.

Ce module centralise l'ensemble des opérations réalisées sur le routeur Cisco. Il établit une connexion SSH sécurisée à l'aide de Netmiko, exécute les commandes de configuration Cisco IOS puis ferme automatiquement la connexion une fois les opérations terminées.

Afin de renforcer la sécurité du projet, les paramètres de connexion du routeur (adresse IP, nom d'utilisateur et mot de passe) sont stockés dans un fichier `.env`. Cette approche évite l'exposition d'informations sensibles dans le code source tout en facilitant le déploiement du projet sur différents environnements.

5.5.3 Création dynamique d'une ACL

Une fonction nommée `block_ip()` a été développée afin de créer automatiquement une liste de contrôle d'accès sur le routeur Cisco.

Cette fonction reçoit en paramètre l'adresse IP source détectée par le moteur Scapy. Elle construit ensuite les commandes Cisco IOS nécessaires à la création d'une règle de filtrage bloquant immédiatement tout trafic provenant de cette adresse [2].

La règle générée adopte le format suivant :

```
deny ip host <SOURCE_IP> any
```

Cette approche permet de transformer automatiquement une détection d'attaque en une action de protection sans intervention de l'administrateur.

La Figure 5.5 présente la création dynamique de l'ACL sur le routeur Cisco.

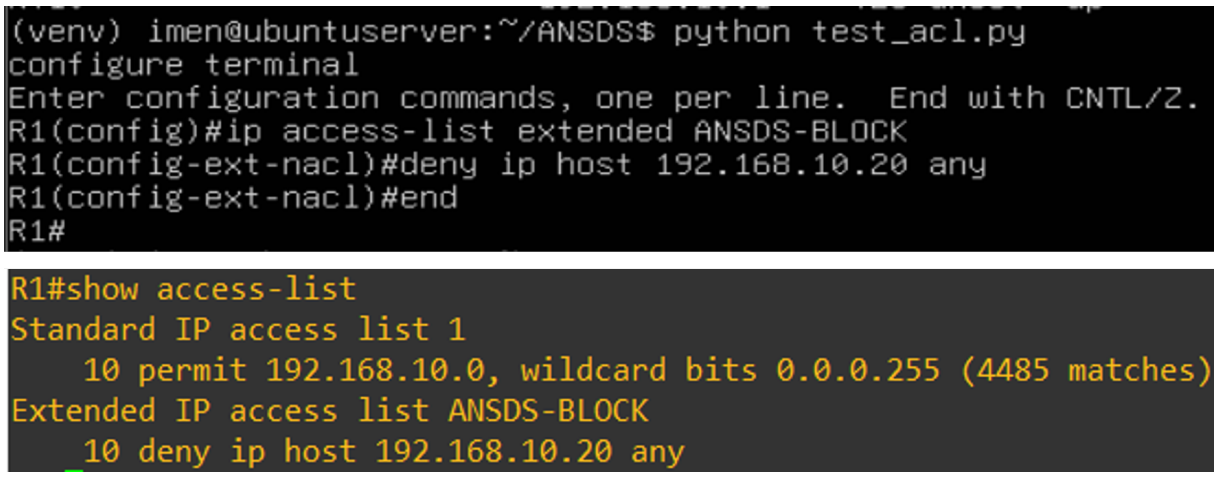


FIGURE 5.5 – Création dynamique d'une ACL avec Netmiko

5.5.4 Suppression automatique d'une ACL

Une seconde fonction, nommée `remove_acl()`, a été développée afin de supprimer automatiquement la liste de contrôle d'accès créée précédemment.

Cette fonctionnalité permet de rétablir rapidement l'état initial du routeur après les phases de validation ou lorsqu'une règle de filtrage n'est plus nécessaire.

La suppression automatique des ACL facilite les campagnes de tests successives et améliore la réutilisabilité du système sans nécessiter de reconfiguration manuelle du routeur.

La Figure 5.6 présente la suppression de l'ACL créée durant les expérimentations.

```
(venv) imen@ubuntuserver:~/ANSDS$ python test_remove_acl.py
configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#no ip access-list extended ANSDS-BLOCK
R1(config)#end
R1#
```

```
R1#show access-list
Standard IP access list 1
    10 permit 192.168.10.0, wildcard bits 0.0.0.255 (4485 matches)
```

FIGURE 5.6 – Suppression automatique d'une ACL

5.6 Gestion des exceptions

Dans un système de réponse automatique, la fiabilité des communications avec les équipements réseau constitue un élément essentiel. Une interruption de la connexion SSH, une erreur d'authentification ou tout autre incident ne doivent pas provoquer l'arrêt du système ANSDS.

Afin d'assurer la robustesse du module Cisco, toutes les opérations réalisées avec Netmiko ont été protégées à l'aide d'une gestion complète des exceptions (`try/except/finally`).

Cette approche permet d'intercepter les erreurs susceptibles de survenir lors de l'établissement de la connexion SSH ou de l'exécution des commandes Cisco IOS, tout en garantissant la poursuite du fonctionnement du moteur de détection.

5.6.1 Gestion des erreurs de communication

Le module Cisco intercepte les principales exceptions pouvant être générées par Netmiko.

Une première exception, `NetmikoTimeoutException`, est déclenchée lorsque le routeur Cisco ne répond pas dans le délai prévu ou devient momentanément indisponible.

Une seconde exception, `NetmikoAuthenticationException`, est levée lorsque les informations d'authentification utilisées pour établir la connexion SSH sont incorrectes [1].

Dans ces deux situations, le module affiche un message explicite permettant d'identifier rapidement l'origine du problème tout en évitant l'arrêt brutal de l'application.

5.6.2 Utilisation des blocs `try`, `except` et `finally`

Toutes les opérations de configuration réalisées par le module Cisco sont encapsulées dans un bloc `try`.

Les différents blocs `except` permettent de traiter séparément les erreurs de communication, les erreurs d'authentification ainsi que toute autre exception inattendue pouvant survenir durant l'exécution du programme.

Enfin, le bloc `finally` garantit la fermeture systématique de la connexion SSH avec le routeur Cisco, indépendamment du résultat des opérations réalisées.

Cette approche permet de libérer correctement les ressources réseau et d'éviter qu'une connexion reste ouverte en cas d'erreur.

5.6.3 Validation du mécanisme de gestion des exceptions

Le bon fonctionnement de la gestion des exceptions a été validé en provoquant volontairement une erreur d'authentification lors de l'établissement de la connexion SSH avec le routeur Cisco.

Le système intercepte correctement l'exception générée par Netmiko, affiche un message d'erreur explicite puis termine proprement son exécution sans provoquer l'arrêt du programme.

Cette validation confirme la robustesse du module Cisco et garantit que le système ANSDS pourra continuer à fonctionner même en présence d'un problème temporaire de communication avec le routeur.

La Figure 5.7 présente le résultat de cette validation.

```
(venv) imen@ubuntuserver:~/ANSDS$ python test_acl.py
Erreur : échec de l'authentification Cisco.
```

FIGURE 5.7 – Validation de la gestion des exceptions Netmiko

5.7 Intégration avec le moteur de détection

Après avoir développé et validé indépendamment le moteur de détection SSH, le module de notification Telegram ainsi que le module Cisco, l'étape suivante consiste à intégrer l'ensemble de ces composants au sein d'une architecture unique de réponse automatique.

L'objectif de cette intégration est de permettre au système ANSDS d'enchaîner automatiquement les différentes opérations depuis la détection d'une attaque jusqu'à l'application d'une contre-mesure sur le routeur Cisco, sans intervention humaine.

Cette approche transforme le système développé en une véritable plateforme automatisée de détection et de réponse aux incidents de sécurité.

5.7.1 Transmission de l'adresse IP détectée

Le moteur de détection développé avec Scapy surveille en permanence le trafic SSH circulant sur le réseau [7].

Lorsqu'une adresse IP dépasse le seuil de tentatives de connexion défini, le détecteur identifie cette adresse comme potentiellement malveillante.

Cette adresse IP est ensuite transmise directement au module Cisco en appelant la fonction `block_ip(ip_src)`.

L'utilisation d'un paramètre permet au module Cisco de rester totalement indépendant du moteur de détection. Ainsi, la même fonction peut être réutilisée ultérieurement avec d'autres mécanismes de détection sans nécessiter de modification du code.

5.7.2 Enchaînement automatique des modules

Une fois le seuil de détection atteint, le système exécute automatiquement les différentes opérations selon une séquence prédéfinie.

Le moteur Scapy détecte tout d'abord l'activité suspecte et construit le message d'alerte contenant les principales informations relatives à l'attaque.

Ce message est ensuite transmis au module Telegram afin d'informer l'administrateur du réseau.

Après l'envoi de cette notification, le module Cisco établit automatiquement une connexion SSH avec le routeur Cisco à l'aide de Netmiko puis crée dynamiquement une ACL permettant de bloquer immédiatement l'adresse IP détectée [1, 2].

Cette séquence assure une transition entièrement automatique entre la phase de détection et la phase de réponse, réduisant ainsi considérablement le temps de réaction face à une attaque.

5.7.3 Architecture finale du système

À l'issue de cette phase, les différents modules développés depuis le début du projet fonctionnent de manière coordonnée afin de constituer une chaîne complète de détection et de réponse automatique.

L'architecture finale du système peut être résumée selon la séquence suivante :

Attaquant
↓
Scapy
↓
Détection SSH
↓
Notification Telegram
↓
Netmiko
↓
Routeur Cisco
↓
ACL dynamique
↓
Blocage automatique de l'adresse IP

Cette architecture modulaire facilite les évolutions futures du projet. Chaque composant peut être amélioré indépendamment tout en conservant une interface claire avec les autres modules du système.

5.8 Validation expérimentale

Après le développement et l'intégration du module Cisco, plusieurs expérimentations ont été réalisées afin de valider progressivement chacune des fonctionnalités développées.

Chaque validation a été effectuée de manière indépendante avant de vérifier le fonctionnement complet de la chaîne de détection et de réponse automatique.

Les expérimentations portent successivement sur la communication avec le routeur Cisco, la création dynamique des ACL, leur suppression, la robustesse du module face aux erreurs de communication ainsi que l'intégration complète avec le moteur de détection développé dans le chapitre précédent.

5.8.1 Validation de la communication avec le routeur Cisco

La première série d'expérimentations consiste à vérifier que le module Netmiko est capable d'établir une connexion SSH avec le routeur Cisco puis d'exécuter automatiquement plusieurs commandes d'administration.

Les résultats obtenus montrent que le système récupère correctement le nom d'hôte du routeur, la version du système Cisco IOS ainsi que les informations relatives aux interfaces réseau.

Cette validation confirme que le module développé peut administrer automatiquement le routeur Cisco et constitue une base fiable pour les opérations de réponse automatique.

La Figure 5.4 illustre les informations récupérées automatiquement lors de cette expérimentation.

5.8.2 Validation de la création dynamique d'une ACL

Une seconde expérimentation a été réalisée afin de vérifier le bon fonctionnement de la fonction `block_ip()`.

Lors de cette validation, le module Cisco établit automatiquement une connexion SSH avec le routeur puis crée une liste de contrôle d'accès nommée `ANSDS-BLOCK`.

L'adresse IP transmise au module est immédiatement intégrée dans une règle de filtrage Cisco IOS, ce qui permet de bloquer automatiquement tout trafic provenant de cette source.

La présence de cette ACL a ensuite été vérifiée directement sur le routeur à l'aide des commandes d'administration Cisco.

La Figure 5.5 présente le résultat obtenu.

5.8.3 Validation de la suppression automatique d'une ACL

Afin de compléter le mécanisme de réponse automatique, une expérimentation supplémentaire a été réalisée pour valider le fonctionnement de la fonction `remove_acl()`.

Cette fonction supprime automatiquement la liste de contrôle d'accès précédemment créée sur le routeur Cisco.

La vérification de la configuration du routeur confirme que l'ACL est correctement supprimée, ce qui permet de retrouver l'état initial du système avant une nouvelle campagne de tests.

La Figure 5.6 présente le résultat de cette validation.

5.8.4 Validation de la robustesse du module Cisco

Le fonctionnement du mécanisme de gestion des exceptions a également été validé.

Pour cette expérimentation, une erreur d'authentification a volontairement été provoquée afin de vérifier la réaction du système.

Les résultats montrent que le module intercepte correctement l'exception générée par Netmiko, affiche un message d'erreur explicite puis termine proprement son exécution sans provoquer l'arrêt de l'application.

Cette validation confirme que le système ANSDS reste opérationnel même lorsqu'une erreur survient durant la communication avec le routeur Cisco.

La Figure 5.7 présente cette expérimentation.

5.8.5 Validation de la chaîne complète de détection et de réponse

La dernière expérimentation consiste à valider le fonctionnement complet du système ANSDS après l'intégration du module Cisco.

Une série de tentatives de connexion SSH est générée depuis la machine Kali Linux afin de dépasser le seuil de détection configuré dans le moteur Scapy.

Lorsque ce seuil est atteint, le système détecte automatiquement l'activité suspecte, construit une notification contenant les principales informations relatives à l'attaque puis transmet cette notification via Telegram.

Après l'envoi de cette alerte, le module Cisco établit automatiquement une connexion SSH avec le routeur et crée une ACL permettant de bloquer immédiatement l'adresse IP de l'attaquant.

Les résultats obtenus démontrent le bon fonctionnement de l'ensemble de la chaîne de traitement développée durant les différentes phases du projet.

La Figure 5.8 présente cette validation finale.

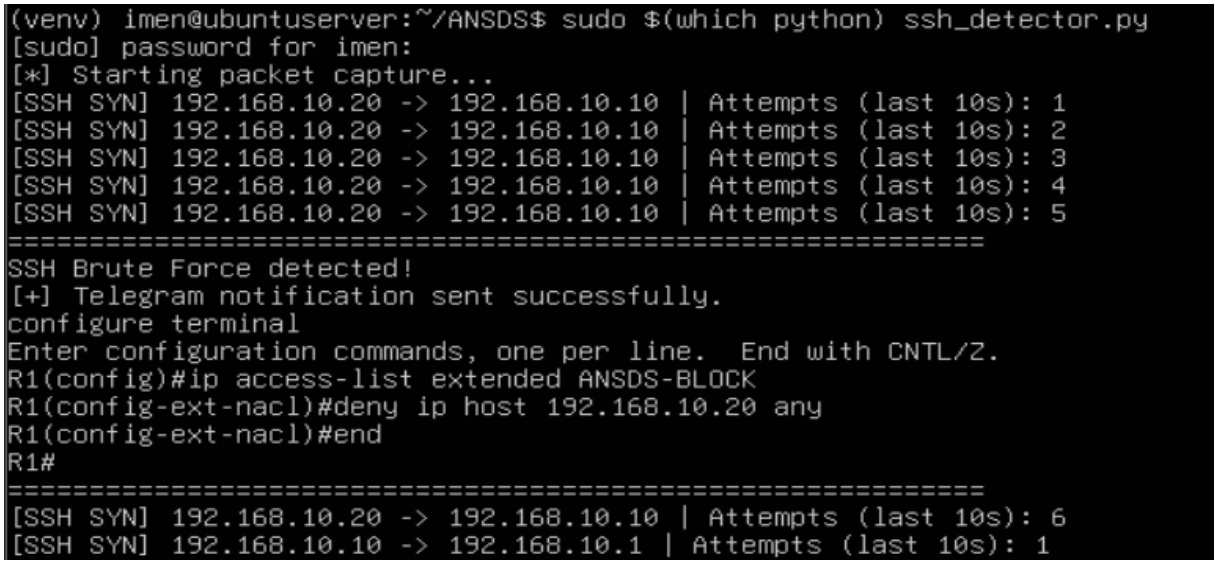


FIGURE 5.8 – Validation complète de la chaîne de détection et de réponse automatique

5.9 Analyse des résultats

Les différentes expérimentations réalisées au cours de cette phase montrent que l'ensemble des objectifs fixés a été atteint.

La préparation du routeur Cisco et la validation des communications SSH ont permis d'établir une infrastructure fiable pour les opérations d'administration automatique. L'utilisation de la bibliothèque Netmiko a ensuite facilité le développement d'un module capable d'exécuter automatiquement des commandes Cisco IOS depuis le serveur Ubuntu [1, 2].

Le module `cisco_acl_manager.py` développé durant cette phase assure la création et la suppression dynamiques des listes de contrôle d'accès tout en conservant une architecture modulaire et facilement réutilisable. La séparation entre les fonctions de détection, de notification et de réponse améliore la lisibilité du code et simplifie les futures évolutions du projet.

Les mécanismes de gestion des exceptions intégrés au module Cisco renforcent également la robustesse du système. Les différentes expérimentations ont montré que les erreurs de communication ou d'authentification sont correctement prises en charge sans provoquer l'arrêt du moteur de détection.

Enfin, l'intégration du module Cisco avec le moteur Scapy et le module Telegram permet désormais d'obtenir une chaîne complète de détection et de réponse automatique [7]. Dès qu'une activité suspecte est détectée, le système génère une alerte, informe l'administrateur puis applique immédiatement une contre-mesure sur le routeur Cisco en créant dynamiquement une ACL de blocage.

Les résultats obtenus démontrent que l'architecture développée répond aux objectifs fixés pour cette phase et constitue une base solide pour les prochaines évolutions du projet ANSDS.

5.10 Conclusion

Au cours de cette quatrième phase, le système ANSDS a été enrichi d'un mécanisme de réponse automatique reposant sur l'administration d'un routeur Cisco à l'aide de la bibliothèque Netmiko.

Les différentes expérimentations ont permis de valider la préparation du routeur pour les connexions SSH, l'installation et la configuration de Netmiko, le développement d'un module Cisco modulaire, la création et la suppression automatiques d'une liste de contrôle d'accès ainsi que la gestion robuste des exceptions.

L'intégration de ce module avec le moteur de détection développé précédemment permet désormais au système d'assurer automatiquement l'ensemble de la chaîne de traitement, depuis la détection d'une attaque SSH jusqu'au blocage immédiat de l'adresse IP malveillante sur le routeur Cisco.

Cette évolution constitue une étape importante dans la réalisation du projet ANSDS puisqu'elle transforme le système en une plateforme capable non seulement de détecter les activités suspectes mais également de réagir automatiquement afin de limiter leur impact sur le réseau.

La phase suivante sera consacrée à l'amélioration du système de réponse automatique et à l'ajout de nouvelles fonctionnalités permettant d'accroître la sécurité, la flexibilité et les capacités d'administration de l'architecture développée.

Chapitre 6

Détection des scans réseau et corrélation des événements

6.1 Introduction

Après la mise en place du mécanisme de détection des attaques SSH, de la notification automatique via Telegram et du blocage dynamique des adresses IP sur le routeur Cisco, la cinquième phase du projet ANSDS vise à améliorer la capacité de détection du système.

La stratégie initiale reposait principalement sur l'observation des paquets TCP SYN destinés au port SSH. Cette approche permet de détecter rapidement des tentatives de connexion, mais un paquet SYN isolé ne constitue pas nécessairement une activité malveillante. Une connexion SSH légitime commence également par l'émission d'un paquet TCP SYN.

La Phase 5 introduit donc une deuxième catégorie d'activité suspecte : les scans réseau réalisés avec Nmap. L'objectif est de détecter une activité de reconnaissance pouvant précéder une tentative d'intrusion.

Les scans étudiés durant cette phase sont :

— TCP SYN Scan (`-sS`) ;
— TCP FIN Scan (`-sF`) ;
— TCP NULL Scan (`-sN`) ;
— TCP Xmas Scan (`-sX`).

La Phase 5 conserve l'architecture modulaire développée lors des phases précédentes et repose sur Scapy pour l'analyse du trafic [7], Python pour la logique de détection, Telegram pour les notifications, Netmiko pour l'automatisation Cisco et les ACL Cisco IOS pour le blocage réseau [1, 2]. Cette architecture correspond à l'évolution prévue du système vers une détection multi-indicateurs.

6.2 Objectifs de la phase

Les objectifs principaux de cette phase sont les suivants :

— détecter les comportements caractéristiques des scans Nmap ;
— prendre en charge plusieurs types de scans TCP ;
— identifier l'adresse IP source du scan ;
— identifier l'adresse IP destination ;
— analyser les drapeaux TCP observés ;

— mesurer le nombre de ports distincts ciblés ;
— utiliser une fenêtre temporelle afin d'identifier un comportement de reconnaissance ;
— réduire les faux positifs liés aux paquets TCP isolés ;
— corréler une activité de scan avec des événements SSH ;
— transmettre une alerte de sécurité via Telegram lorsqu'une activité suspecte est confirmée ;
— appliquer automatiquement un blocage temporaire sur le routeur Cisco ;
— journaliser les événements importants du système ;
— supprimer automatiquement le blocage après expiration de la durée configurée.

La conception de la Phase 5 suit une approche progressive afin de conserver la modularité des composants existants et d'éviter de rendre le moteur de détection SSH inutilement complexe.

6.3 Évolution de la stratégie de détection

6.3.1 Limites de la détection SSH initiale

Lors des phases précédentes, le moteur de détection surveillait les paquets TCP SYN destinés au port 22.

Cette méthode présente un avantage important : elle permet d'identifier une tentative de connexion dès le début de l'établissement de la connexion TCP.

Cependant, un paquet SYN isolé peut également correspondre à une activité parfaitement légitime.

La détection doit donc prendre en compte le comportement global de la source plutôt qu'un paquet unique.

6.3.2 Introduction de la détection Nmap

La Phase 5 étend le moteur afin d'identifier les comportements associés aux scans Nmap.

Un scan réseau se caractérise notamment par l'envoi rapide de paquets vers plusieurs ports de destination.

Le détecteur analyse donc plusieurs informations :
— l'adresse IP source ;
— l'adresse IP destination ;
— les drapeaux TCP ;
— les ports de destination ;
— le nombre de ports distincts ;
— la fenêtre temporelle.

Le principe général retenu est le suivant :

Même source IP
↓
Même signature TCP
↓
Plusieurs ports distincts
↓
Courte fenêtre temporelle

↓
Événement de scan

Cette approche permet d'éviter de considérer un paquet TCP isolé comme un scan.

6.4 Signatures TCP des scans Nmap

Les différents types de scans étudiés utilisent des combinaisons différentes de drapeaux TCP [7].

6.4.1 TCP SYN Scan

Le SYN Scan est réalisé avec l'option :

```
nmap -sS
```

Les paquets observés contiennent principalement le drapeau TCP :

```
SYN
```

Le détecteur associe ces paquets à l'adresse IP source et aux ports de destination observés.

```
(venv) imen@ubuntuserver:~/ANSDS$ sudo $(which python) ~/ANSDS/nmap_detector.py
[*] Starting Nmap scan detection...
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 1
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 2
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 3
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 4
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 5
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 6
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 7
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 8
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 9
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 10
============================================================
Nmap scan detected!
Scan Type       : NMAP_SYN_SCAN
Source IP       : 192.168.10.20
Destination IP  : 192.168.10.10
Distinct Ports  : 10
Time Window     : 2 seconds
============================================================
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 11
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 12
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 13
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 14
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 15
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 16
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 17
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 18
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 19
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 20
```

FIGURE 6.1 – Détection d'un scan Nmap SYN

6.4.2 TCP FIN Scan

Le FIN Scan est réalisé avec :

```
nmap -sF
```

La signature observée correspond à un paquet TCP contenant le drapeau :

```
FIN
```

Le détecteur identifie cette signature puis vérifie si elle est répétée vers plusieurs ports dans une courte période.

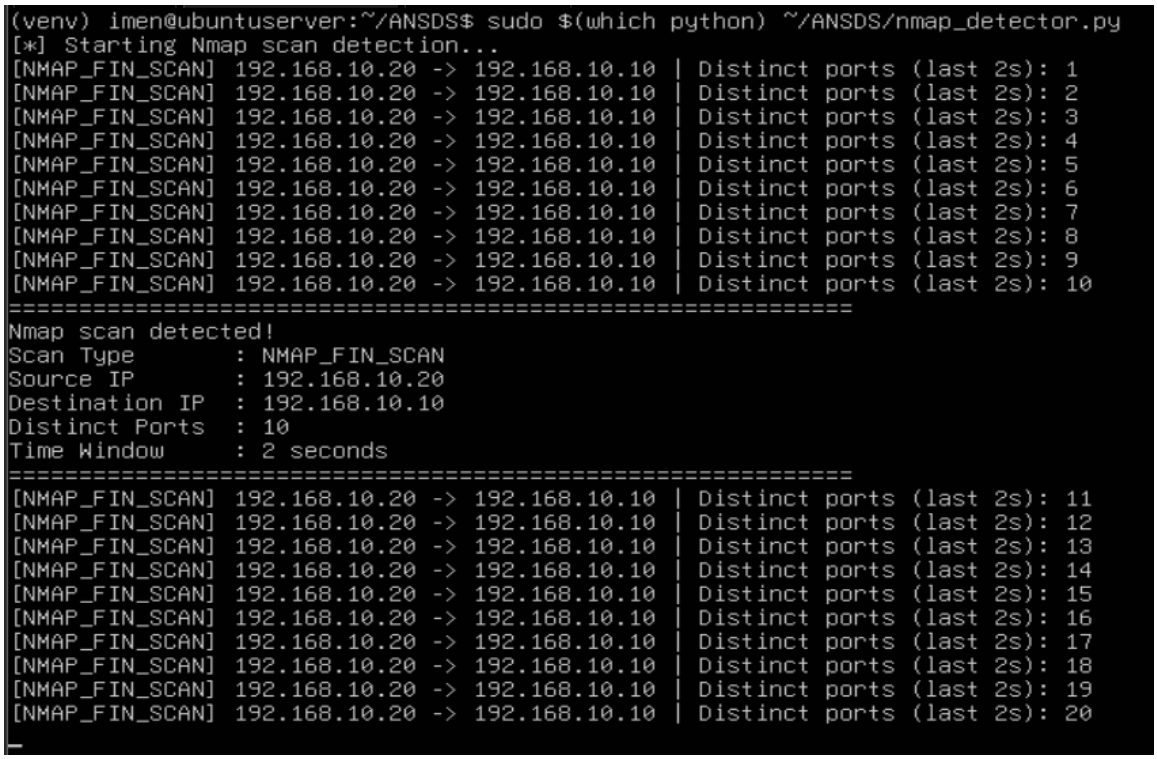


FIGURE 6.2 – Détection d'un scan Nmap FIN

6.4.3 TCP NULL Scan

Le NULL Scan est réalisé avec :

```
nmap -sN
```

Dans ce cas, aucun des principaux drapeaux TCP observés dans les autres signatures n'est activé.

La signature considérée par le détecteur correspond donc à l'absence de drapeaux TCP.

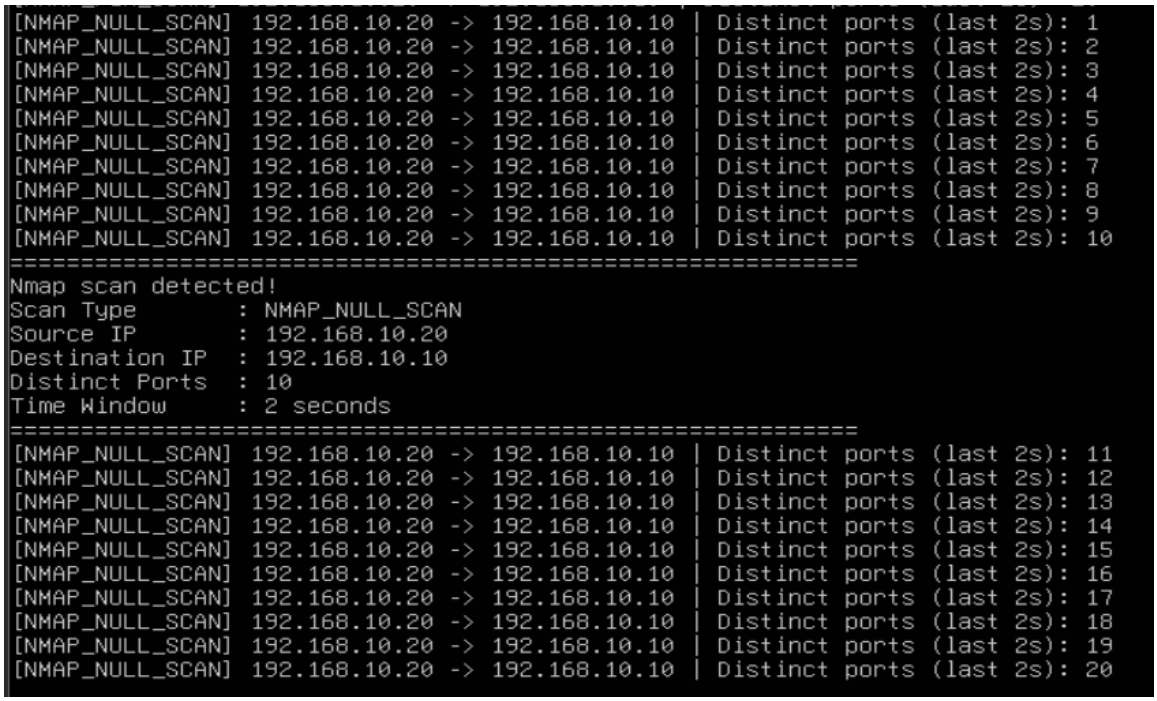


FIGURE 6.3 – Détection d'un scan Nmap NULL

6.4.4 TCP Xmas Scan

Le Xmas Scan est réalisé avec :

```
nmap -sX
```

La signature observée correspond à la combinaison de plusieurs drapeaux TCP :

```
FIN + PSH + URG
```

Cette combinaison permet au détecteur de distinguer ce comportement des autres signatures étudiées.

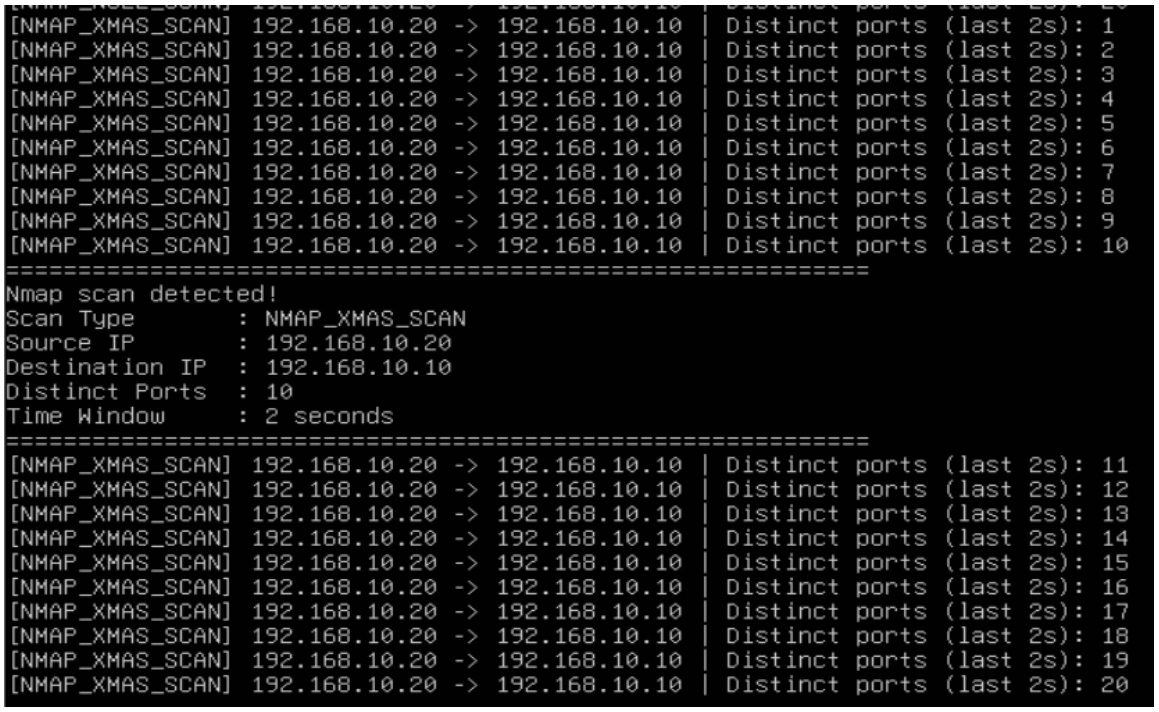


FIGURE 6.4 – Détection d'un scan Nmap Xmas

6.5 Détection comportementale des scans

La détection ne repose pas uniquement sur la présence d'une signature TCP.

Un paquet isolé peut correspondre à un comportement légitime ou à une communication réseau normale. Pour cette raison, le détecteur conserve les informations relatives aux paquets observés et analyse leur comportement dans une fenêtre temporelle limitée.

Pour une même adresse IP source, le système regroupe les paquets présentant une signature similaire et enregistre les ports de destination ciblés.

Lorsque plusieurs ports distincts sont observés sur une courte période, le comportement devient compatible avec une activité de reconnaissance.

Le mécanisme peut être représenté comme suit :

Source IP
↓
Signature TCP
↓
Ports de destination
↓
Nombre de ports distincts
↓
Fenêtre temporelle
↓
Détection Nmap

Cette logique permet de différencier une activité de scan d'un paquet TCP isolé.

6.6 Réduction des faux positifs

La réduction des faux positifs constitue un objectif important de cette phase.

Un système de détection réseau ne doit pas générer une alerte simplement parce qu'un paquet possède une signature particulière.

Un trafic légitime peut contenir des paquets TCP présentant des caractéristiques similaires à celles utilisées par un détecteur.

Le système considère donc plusieurs éléments simultanément :

— la source du trafic ;
— la signature TCP ;
— le nombre de ports ciblés ;
— la rapidité des requêtes ;
— la présence éventuelle d'une activité SSH associée.

6.6.1 Validation du trafic légitime

Un scénario de trafic légitime a été utilisé afin de vérifier que le détecteur ne considère pas automatiquement toute activité TCP comme un scan Nmap.

La capture suivante présente le trafic utilisé durant cette validation.

```
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 1
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 2
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 3
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 4
[NMAP_SYN_SCAN] 192.168.10.20 -> 192.168.10.10 | Distinct ports (last 2s): 5
```

FIGURE 6.5 – Trafic réseau légitime observé durant la validation

Cette validation permet de vérifier que la décision de détection repose sur le comportement global observé plutôt que sur un paquet isolé.

6.7 Corrélation avec l'activité SSH

La détection d'un scan Nmap constitue un indicateur de reconnaissance, mais elle ne permet pas à elle seule de conclure qu'une tentative d'intrusion SSH est en cours.

La Phase 5 introduit donc une logique de corrélation entre l'activité de reconnaissance et l'activité SSH.

Le principe est le suivant :

Nmap Scan détecté
↓
Surveillance de l'activité SSH
↓
Évidence d'authentification échouée
↓
Corrélation confirmée
↓
Événement de sécurité

Cette approche permet de renforcer la décision de sécurité en combinant plusieurs indicateurs plutôt qu'en utilisant une seule observation réseau.

La corrélation entre le scan et l'activité SSH a été validée expérimentalement.

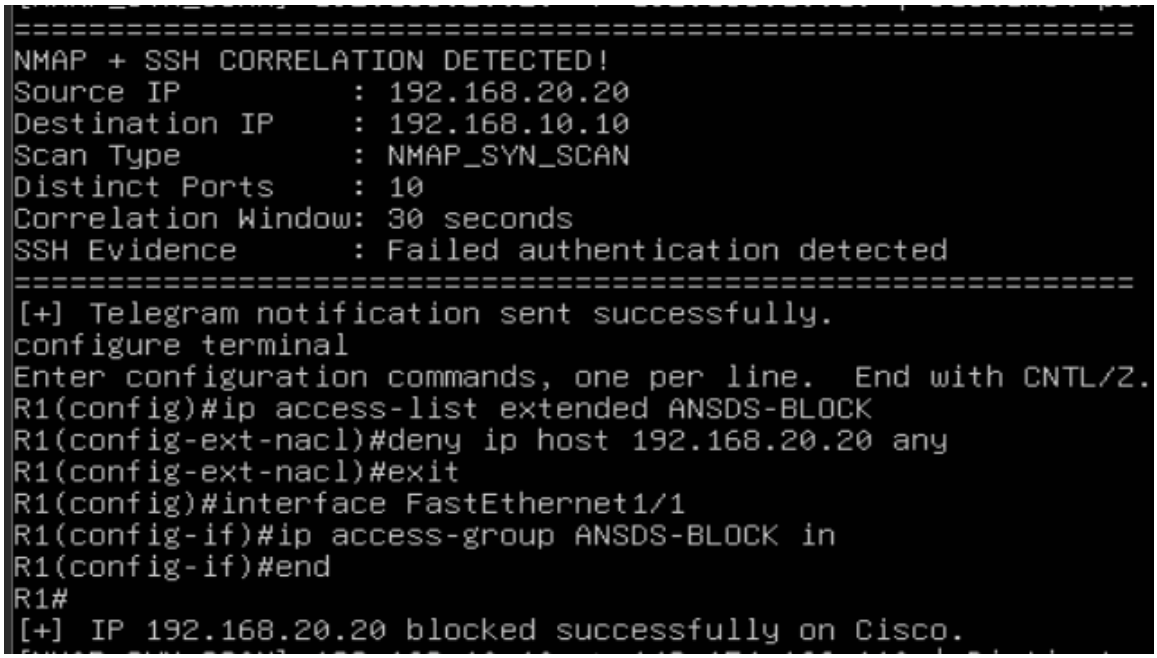


FIGURE 6.6 – Corrélation entre un scan Nmap et une activité SSH suspecte

6.7.1 Scan sans corrélation SSH

Un deuxième scénario a été utilisé afin de vérifier qu'un scan Nmap seul ne déclenche pas automatiquement la même réponse qu'un événement corrélé avec une activité SSH.

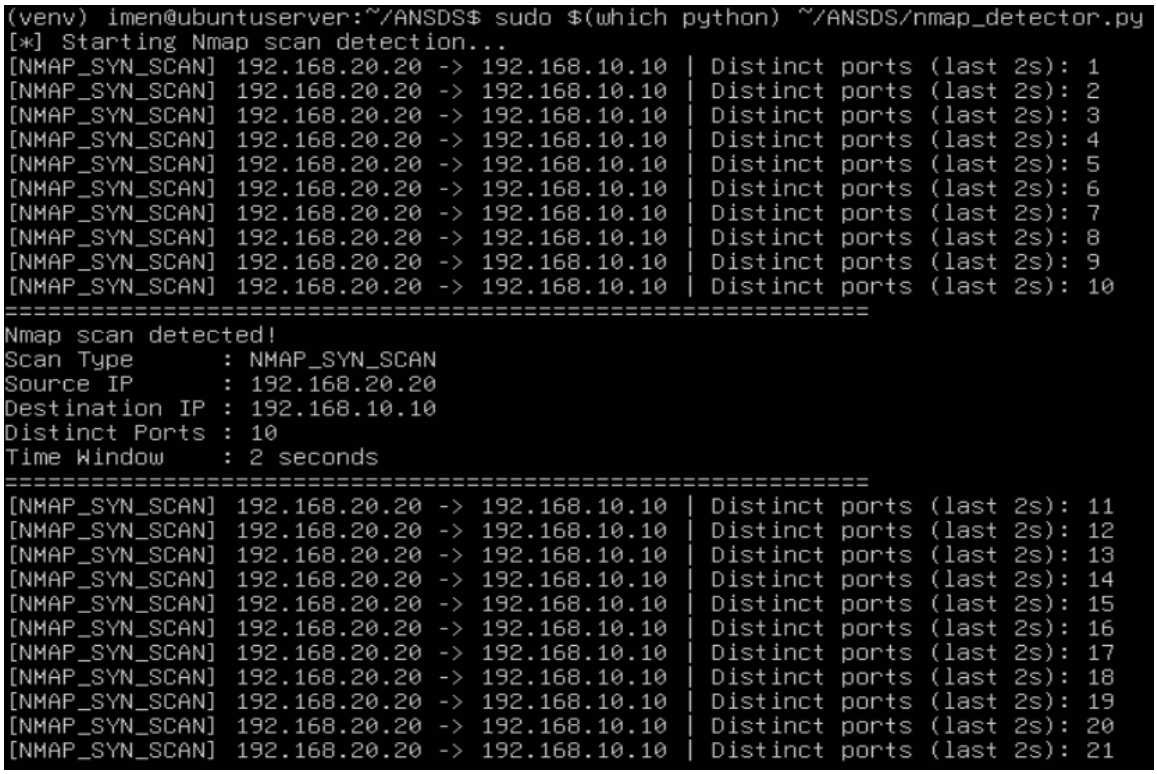


FIGURE 6.7 – Scan Nmap sans corrélation avec une activité SSH

Cette séparation permet de distinguer une activité de reconnaissance isolée d'un comportement combinant reconnaissance et tentative d'accès au service SSH.

6.8 Notification de sécurité

Lorsqu'une corrélation est confirmée, le système transmet un événement au module de notification Telegram.

Le module Telegram reste séparé du moteur de détection afin de conserver une architecture modulaire.

Le workflow est alors :

Nmap Detection
↓
SSH Correlation
↓
Security Event
↓
Telegram Notification

La notification permet à l'administrateur d'être informé de l'activité détectée avant ou pendant l'application de la mesure de protection réseau.

6.9 Réponse automatique Cisco

Après confirmation de l'événement de sécurité, le système peut déclencher le mécanisme de réponse automatique développé durant la phase précédente.

Le module Cisco utilise Netmiko afin d'établir une connexion SSH avec le routeur Cisco et d'appliquer dynamiquement une ACL [1, 2].

La règle de blocage utilisée est :

```
deny ip host \<SOURCE\_IP> any
```

Cette approche permet de transformer automatiquement un événement de sécurité confirmé en mesure de protection réseau.

6.9.1 Activation du blocage temporaire

Contrairement à un blocage permanent, le mécanisme développé durant cette phase utilise une durée de blocage limitée.

Lorsqu'un événement est confirmé, le système applique le blocage puis programme automatiquement son expiration.

Le cycle de traitement est :

Détection
↓
Corrélation
↓
Notification
↓
Blocage Cisco
↓
Expiration
↓
Déblocage automatique

La capture suivante présente l'activation du blocage temporaire.

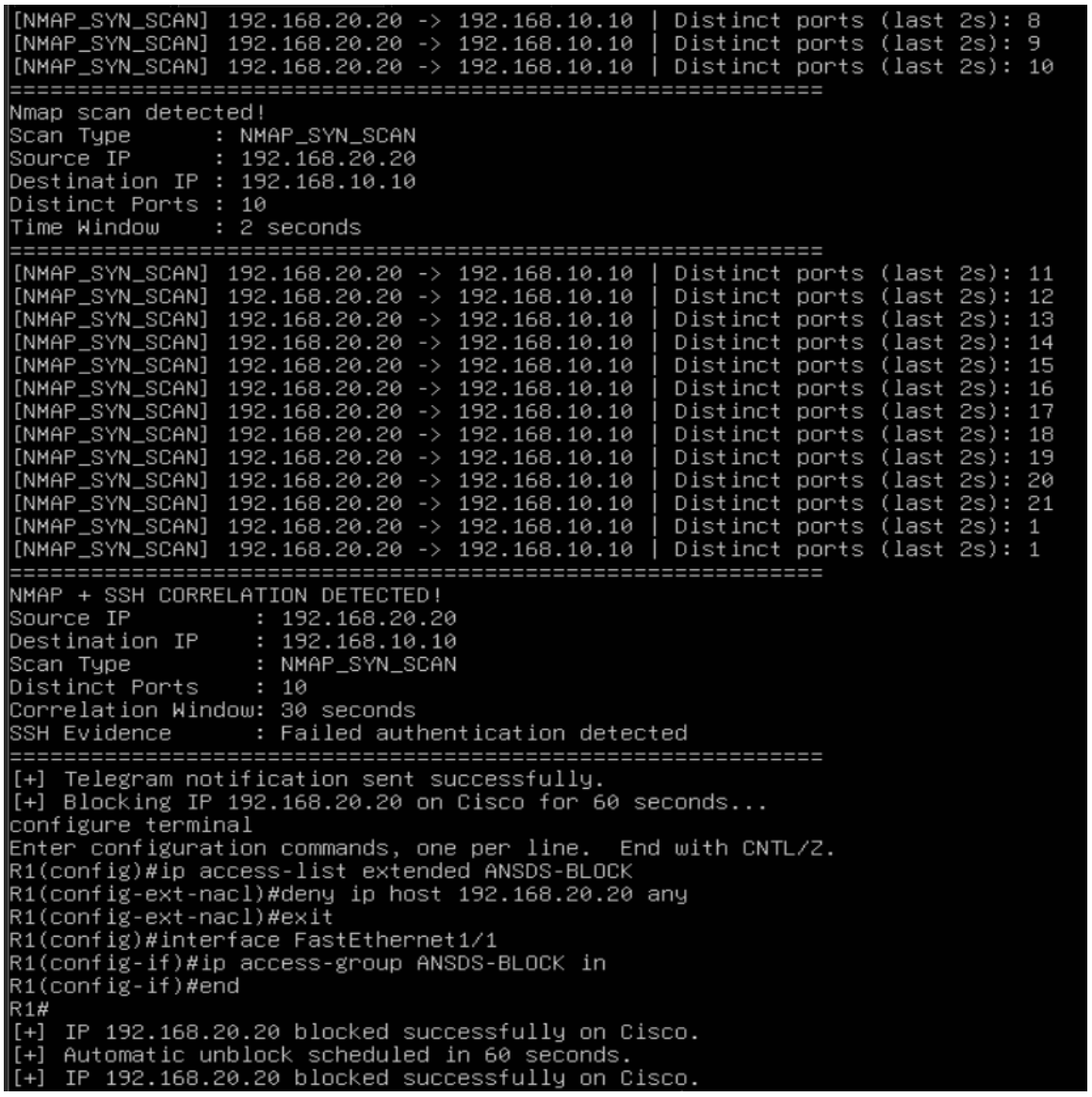


FIGURE 6.8 – Activation du blocage temporaire

6.9.2 Application de l'ACL Cisco

Le module Cisco crée dynamiquement l'ACL destinée à bloquer l'adresse IP source. L'application de cette règle est réalisée automatiquement à travers Netmiko [1, 2].

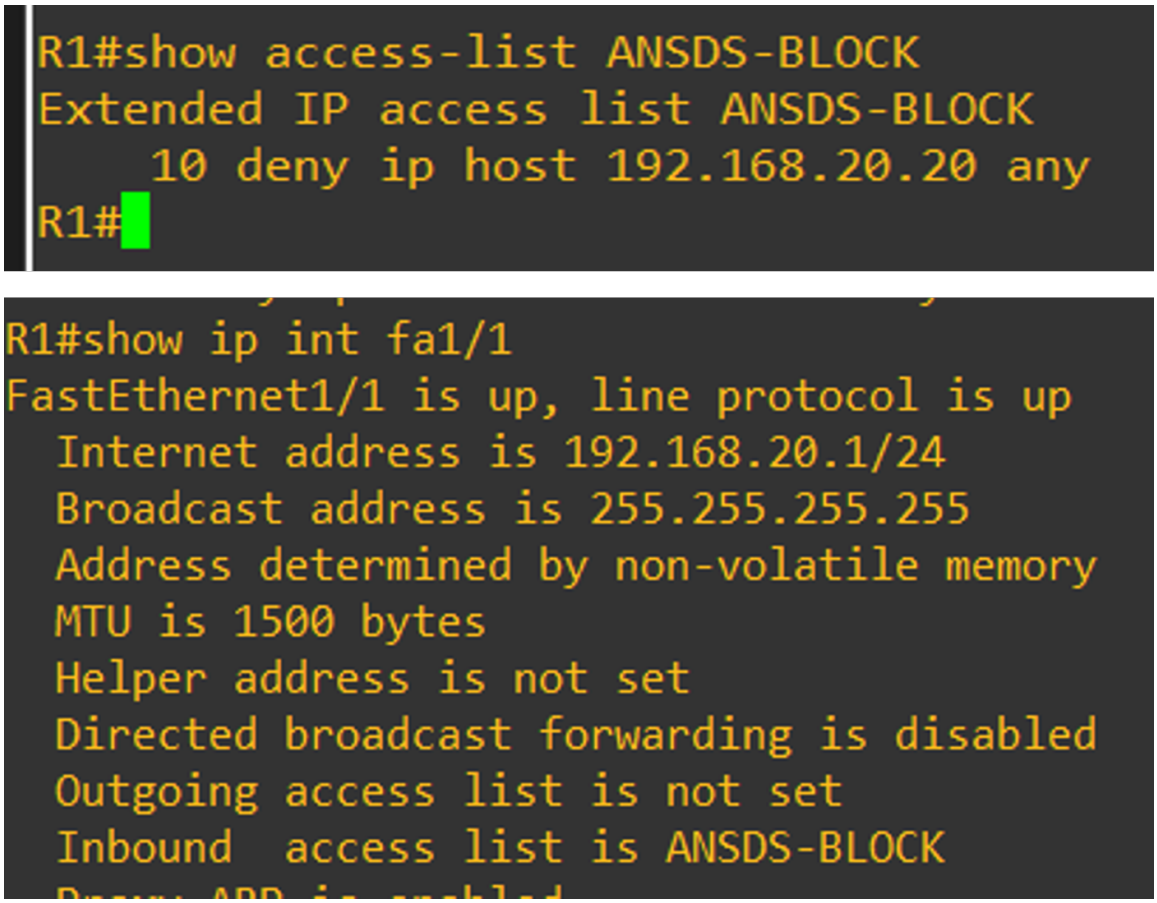


FIGURE 6.9 – Application de l'ACL de blocage sur le routeur Cisco

Cette étape confirme l'intégration entre le moteur de détection et l'infrastructure réseau.

6.10 Journalisation des événements

La Phase 5 introduit également un mécanisme de journalisation permettant de conserver une trace des événements de sécurité.

Le système journalise notamment :

— l'adresse IP source ;
— l'adresse IP destination ;
— le type de scan ;
— le nombre de ports distincts ;
— la fenêtre temporelle ;
— la confirmation de la corrélation SSH ;
— l'envoi de la notification Telegram ;
— l'application du blocage Cisco ;
— la durée du blocage ;
— l'expiration du blocage ;
— la suppression de l'ACL.

Le journal permet ainsi de reconstruire le cycle de traitement d'un événement de sécurité et facilite la validation du fonctionnement du système.

6.11 Expiration automatique du blocage

Le blocage appliqué par le système est temporaire.

À l'expiration de la durée configurée, le module Cisco supprime automatiquement la règle créée précédemment.

Cette fonctionnalité évite qu'une ACL temporaire reste active indéfiniment et permet au réseau de revenir automatiquement à son état initial.

Le cycle complet est donc :

****Detection****
→
****Correlation****
→
****Alert****
→
****Block****
→
****Expiration****
→
****Unblock****

6.11.1 Suppression de l'ACL

La suppression automatique de l'ACL est réalisée lorsque la durée de blocage arrive à expiration.

```
R1#show access-list ANSDS-BLOCK

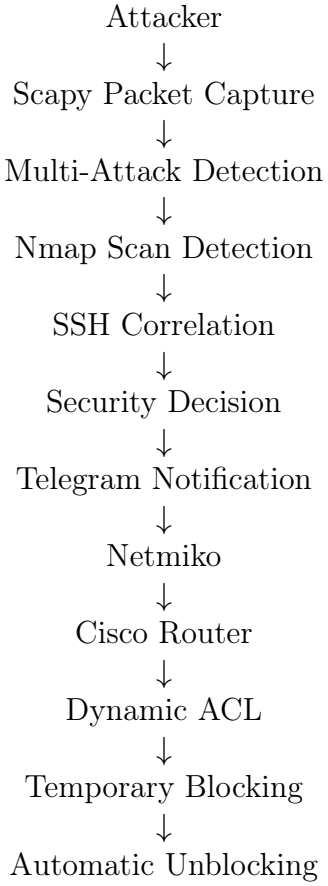
R1#show ip int fa1/1
FastEthernet1/1 is up, line protocol is up
  Internet address is 192.168.20.1/24
  Broadcast address is 255.255.255.255
  Address determined by non-volatile memory
  MTU is 1500 bytes
  Helper address is not set
  Directed broadcast forwarding is disabled
  Outgoing access list is not set
  Inbound  access list is not set
```

FIGURE 6.10 – Suppression automatique de l'ACL après expiration du blocage

Cette fonctionnalité complète le cycle de vie du blocage temporaire.

6.12 Architecture finale de la Phase 5

À l'issue de cette phase, l'architecture du système peut être représentée par la chaîne suivante :



Cette architecture conserve la séparation des responsabilités entre les différents modules du projet.

Le moteur de détection est chargé de l'analyse du trafic, le module Telegram assure la notification, le module Cisco réalise la réponse réseau et le module de journalisation assure la traçabilité des événements.

6.13 Validation expérimentale

Une série d'expérimentations a été réalisée afin de valider progressivement les fonctionnalités introduites durant cette phase.

Les tests ont porté sur les éléments suivants :

— détection d'un Nmap SYN Scan ;
— détection d'un Nmap FIN Scan ;
— détection d'un Nmap NULL Scan ;
— détection d'un Nmap Xmas Scan ;
— observation d'un trafic légitime ;
— corrélation entre une activité Nmap et une activité SSH ;
— absence de corrélation lorsqu'aucune activité SSH correspondante n'est observée ;
— transmission de la notification Telegram ;

— activation du blocage temporaire ;
— application de l'ACL Cisco ;
— expiration du blocage ;
— suppression automatique de l'ACL.

Les captures présentées dans ce chapitre constituent les éléments de preuve expérimentale associés à ces différents scénarios.

6.14 Analyse des résultats

Les expérimentations montrent que l'architecture ANSDS peut être étendue à plusieurs catégories d'événements réseau sans remettre en cause les composants développés durant les phases précédentes.

L'ajout de la détection Nmap permet au système d'observer une activité de reconnaissance en complément de la surveillance des tentatives SSH.

L'utilisation de plusieurs indicateurs améliore également la prise de décision. Une signature TCP isolée n'est pas considérée comme suffisante pour déclencher une réponse complète. Le comportement est analysé à travers les ports ciblés, la répétition des paquets, la fenêtre temporelle et, lorsque cela est pertinent, l'activité SSH associée.

La corrélation permet ainsi de distinguer une activité de reconnaissance isolée d'une séquence plus significative combinant scan réseau et tentative d'accès SSH.

L'intégration avec Telegram et Netmiko permet ensuite de conserver la chaîne automatisée développée lors des phases précédentes.

Enfin, le blocage temporaire et son expiration automatique permettent de gérer le cycle complet d'une mesure de protection réseau sans intervention manuelle.

6.15 Journalisation et traçabilité

Le système conserve désormais les événements importants du traitement dans le fichier de journalisation ANSDS.

Les événements permettent notamment de suivre :

```
NMAP\_DETECTED

SSH\_CORRELATION\_CONFIRMED

TELEGRAM\_ALERT\_SENT

IP\_BLOCKED

TEMPORARY\_BLOCK\_SCHEDULED

TEMPORARY\_BLOCK\_EXPIRED

IP\_UNBLOCKED
```

Cette journalisation fournit une trace chronologique des décisions prises par le système et permet de vérifier que les différents composants ont correctement exécuté leur rôle.

6.16 Conclusion

Cette phase a permis d'étendre le système ANSDS au-delà de la simple détection des tentatives SSH.

Un mécanisme de détection des scans Nmap a été intégré afin d'identifier plusieurs comportements de reconnaissance réseau à partir des signatures TCP observées avec Scapy [7].

Les quatre types de scans étudiés sont les scans SYN, FIN, NULL et Xmas.

La détection repose sur l'analyse comportementale de plusieurs paquets et de plusieurs ports plutôt que sur l'observation d'un paquet isolé.

La Phase 5 introduit également une logique de corrélation avec l'activité SSH. Cette approche permet de renforcer la décision de sécurité en combinant les informations provenant de plusieurs indicateurs.

Lorsque l'événement est confirmé, le système peut transmettre automatiquement une notification Telegram puis appliquer une ACL dynamique sur le routeur Cisco à l'aide de Netmiko [1, 2].

Le blocage est temporaire et son expiration entraîne automatiquement la suppression de l'ACL.

Enfin, la journalisation fournit une traçabilité des différentes étapes du traitement.

La chaîne obtenue peut être résumée ainsi :

****Network Reconnaissance****
→
****Nmap Detection****
→
****SSH Correlation****
→
****Security Alert****
→
****Telegram****
→
****Cisco ACL****
→
****Temporary Block****
→
****Automatic Unblock****

Cette évolution constitue une étape supplémentaire vers un système ANSDS capable de détecter et de traiter automatiquement plusieurs comportements malveillants au niveau réseau.

Chapitre 7

Conclusion générale et perspectives

7.1 Bilan général du projet

Le projet Automated Network Self-Defense System (ANSDS) avait pour objectif de concevoir et de développer progressivement un système capable d'analyser le trafic réseau, d'identifier des comportements suspects, de transmettre des alertes de sécurité et d'appliquer automatiquement une réponse sur l'infrastructure réseau.

La réalisation du projet a été menée dans un environnement expérimental construit sous GNS3. Celui-ci intègre un routeur Cisco, un commutateur, un serveur Ubuntu et une machine Kali Linux. Cette infrastructure a permis de reproduire différents scénarios réseau dans un environnement contrôlé et de valider progressivement les fonctionnalités développées.

Le développement a suivi une approche incrémentale. Le projet a d'abord porté sur l'observation et l'analyse du trafic SSH, avant de mettre en place un moteur de détection avec Scapy. Celui-ci a ensuite été enrichi par un mécanisme de notification Telegram et par l'automatisation du routeur Cisco à l'aide de Netmiko.

La dernière phase a étendu la capacité de détection aux activités de reconnaissance réseau réalisées avec Nmap. Plusieurs signatures TCP ont été prises en compte, notamment les scans SYN, FIN, NULL et Xmas. La décision de sécurité a été renforcée par l'analyse du comportement de la source, du nombre de ports ciblés, de la répétition des paquets, de la temporalité des événements et de la corrélation avec une activité SSH suspecte.

À l'issue du développement, le système est capable d'enchaîner plusieurs étapes de manière automatisée : détection, analyse, corrélation, notification, blocage temporaire sur le routeur Cisco et suppression automatique de la règle de blocage après expiration de la durée configurée.

7.2 Principales contributions réalisées

Les principales contributions du projet peuvent être regroupées autour de plusieurs fonctionnalités complémentaires.

- Mise en place d'un laboratoire réseau expérimental sous GNS3 permettant de reproduire et de contrôler différents scénarios de sécurité.
- Analyse du trafic SSH et identification des paquets TCP SYN destinés au port 22.
- Développement d'un moteur de détection en Python avec Scapy reposant notamment sur un comptage des tentatives et une fenêtre temporelle glissante.

— Mise en place d'un système de notification automatique avec Telegram permettant de transmettre les informations principales associées à un événement détecté.
— Développement d'un mécanisme d'administration automatique du routeur Cisco avec Netmiko.
— Création dynamique d'ACL permettant de bloquer l'adresse IP identifiée comme source d'une activité suspecte.
— Mise en place d'un mécanisme de suppression automatique des ACL après expiration du blocage temporaire.
— Intégration d'une gestion des exceptions afin de renforcer la robustesse du module d'administration Cisco.
— Extension du moteur de détection aux scans Nmap SYN, FIN, NULL et Xmas.
— Mise en place d'une logique de corrélation entre une activité de reconnaissance réseau et une activité SSH suspecte.
— Ajout d'un mécanisme de journalisation permettant de conserver une trace chronologique des événements et des décisions prises par le système.
— Validation expérimentale progressive des composants individuels puis de la chaîne complète de détection et de réponse automatique.

7.3 Bilan de la chaîne de défense développée

L'architecture obtenue à l'issue du projet repose sur une séparation claire des responsabilités entre les différents composants.

Le moteur Scapy assure la capture et l'analyse du trafic réseau. La logique de détection identifie les comportements correspondant aux scénarios étudiés. Le module Telegram assure la transmission des alertes de sécurité. Netmiko permet d'établir la communication avec le routeur Cisco, tandis que le module de gestion des ACL applique et supprime dynamiquement les règles de blocage.

La chaîne finale peut être résumée comme suit :

Détection → Analyse comportementale → Corrélation → Décision de sécurité → Notification Telegram → Netmiko → ACL Cisco → Blocage temporaire → Déblocage automatique

Cette organisation modulaire constitue un élément important du système. Elle permet de faire évoluer les mécanismes de détection et de réponse sans remettre en cause l'ensemble de l'architecture.

7.4 Limites du système

Malgré les résultats obtenus, la solution développée présente certaines limites liées à son caractère expérimental.

Premièrement, les différentes fonctionnalités ont été développées et validées dans un laboratoire réseau contrôlé sous GNS3. Les résultats obtenus doivent donc être considérés dans le contexte de cet environnement expérimental et ne constituent pas à eux seuls une validation dans une infrastructure de production.

Deuxièmement, les mécanismes de détection reposent sur des signatures et des critères comportementaux définis pour les scénarios étudiés. Les seuils, les fenêtres temporelles et

les critères de corrélation peuvent donc nécessiter une adaptation lorsqu'ils sont utilisés dans un environnement présentant des caractéristiques différentes.

Troisièmement, la couverture des comportements malveillants reste limitée aux scénarios étudiés dans le cadre du projet. Le système traite principalement les tentatives suspectes visant SSH ainsi que plusieurs types de scans réseau Nmap.

Enfin, la réponse automatique développée repose sur l'administration d'un équipement Cisco compatible avec le mécanisme d'automatisation mis en place. Une généralisation à d'autres équipements nécessiterait l'adaptation du module de réponse.

Ces limites ne remettent pas en cause les objectifs atteints dans le cadre du stage, mais définissent clairement le périmètre actuel de la solution.

7.5 Perspectives d'évolution

Plusieurs évolutions peuvent être envisagées afin d'améliorer les capacités du système ANSDS.

Une première évolution consisterait à étendre la détection à d'autres types de trafic et de comportements réseau. Le moteur pourrait notamment intégrer de nouvelles signatures et de nouveaux scénarios d'attaque afin d'élargir la couverture du système.

Une deuxième évolution concernerait l'amélioration de la corrélation des événements. Plusieurs indicateurs provenant de différentes sources pourraient être combinés afin d'obtenir une décision de sécurité plus précise et de réduire davantage les faux positifs.

Le système pourrait également évoluer vers un mécanisme de classification ou de score de risque permettant d'associer différents niveaux de gravité aux événements détectés. La réponse pourrait alors être adaptée au niveau de menace identifié.

Une autre perspective serait l'intégration d'une interface de supervision permettant de visualiser les événements détectés, les adresses bloquées, les alertes transmises et l'historique des actions réalisées par le système.

Enfin, une évolution vers une architecture intégrée à une plateforme de supervision de sécurité ou à un SIEM permettrait de centraliser les événements et de faciliter l'analyse globale des incidents.

7.6 Bilan du stage

La réalisation de ce projet a permis de mettre en pratique plusieurs connaissances acquises dans les domaines des réseaux, de la programmation Python, de la cybersécurité et de l'administration des équipements réseau.

Le travail réalisé a notamment permis d'approfondir l'utilisation de GNS3, Cisco IOS, Wireshark, Python, Scapy et Netmiko. Il a également permis de comprendre les différentes étapes nécessaires à la construction progressive d'un mécanisme de détection et de réponse automatisée.

Au-delà de l'implémentation technique, le projet a également permis d'adopter une démarche expérimentale basée sur l'observation du trafic, la formulation d'hypothèses, la réalisation de scénarios contrôlés, l'analyse des résultats et la validation progressive des fonctionnalités.

Le projet ANSDS constitue ainsi une mise en application concrète des principes de détection et de réponse automatisée dans un environnement réseau contrôlé. Les résultats

obtenus fournissent une base fonctionnelle et modulaire pouvant être enrichie par de futurs mécanismes de détection, de corrélation et de réponse.

Bibliographie

[1] Kirk Byers. Netmiko documentation. https://ktbyers.github.io/netmiko/. Consulté le 18 août 2026.

[2] Cisco Systems. Cisco ios documentation. https://www.cisco.com/. Consulté le 18 août 2026.

[3] GNS3 Technologies. Gns3 documentation. https://docs.gns3.com/. Consulté le 18 août 2026.

[4] Information Sciences Institute. Transmission control protocol. https://www.rfc-editor.org/rfc/rfc793. RFC 793, 1981. Consulté le 18 août 2026.

[5] Nmap Project. Nmap reference guide. https://nmap.org/book/man.html. Consulté le 18 août 2026.

[6] Python Software Foundation. Python documentation. https://docs.python.org/3/. Consulté le 18 août 2026.

[7] Scapy Project. Scapy documentation. https://scapy.readthedocs.io/. Consulté le 18 août 2026.

[8] Telegram. Telegram bot api. https://core.telegram.org/bots/api. Consulté le 18 août 2026.

[9] Wireshark Foundation. Wireshark documentation. https://www.wireshark.org/docs/. Consulté le 18 août 2026.

[10] T. Ylonen and C. Lonvick. The secure shell (ssh) transport layer protocol. https://www.rfc-editor.org/rfc/rfc4253. RFC 4253, 2006. Consulté le 18 août 2026.